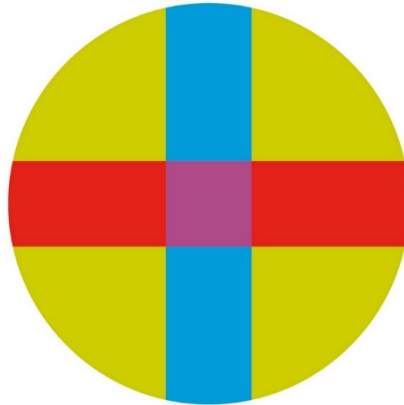


UNIVERSIDAD CEU - SAN PABLO

ESCUELA POLITÉCNICA SUPERIOR

GRADO EN INGENIERÍA DE SISTEMAS DE TELECOMUNICACIÓN



TRABAJO FIN DE GRADO

**Diseño y desarrollo de librería de
AutoML para la optimización
automática de modelos de
clasificación**

Autor: Víctor Vega Martínez
Director: Abraham Otero Quintana

Julio 2025



Datos del alumno

NOMBRE:

Datos del Trabajo

TÍTULO DEL PROYECTO:

Tribunal calificador

PRESIDENTE:

FDO.:

SECRETARIO:

FDO.:

VOCAL:

FDO.:

Reunido este tribunal el ____/____/____, acuerda otorgar al Trabajo Fin de Grado
presentado por Don _____ la calificación de _____.

AGRADECIMIENTOS

En primer lugar, quiero expresar mi más profundo agradecimiento a mi tutor, Abraham Otero Quintana, cuyo conocimiento, dedicación y orientación han sido fundamentales para la realización de este trabajo; sin su apoyo experto, este proyecto no habría sido posible.

Agradezco también a mi familia y a mis seres queridos por su constante aliento y comprensión a lo largo de todo el proceso. Vuestra paciencia y apoyo incondicional han sido mi fuerza en los momentos de mayor dificultad.

Finalmente, deseo dar las gracias a mis amigos por las valiosas conversaciones y estímulos que me han ayudado a mantener la motivación, así como a esa persona especial que comparte día a día mis metas y alegrías, por estar siempre ahí y hacerme creer en mí mismo.

ABSTRACT

Artificial neural networks are powerful computational models, but they require the tuning of numerous hyperparameters to achieve good performance. This process is both costly and complex, and it is called AutoML. Currently, there is no AutoML library on top of Keras oriented towards generic classification problems.

A heuristic has been designed to iteratively set the different hyperparameters, trying to minimize the computational cost of their selection. This heuristic has been implemented in a Python library, and it has been validated using real datasets. The evolution of the generated model's performance has been analyzed by increasing the computational cost of the resources used by the library, comparing the final performance with the results in the literature. This process has enabled finding the balance between the computational cost required by the library and the performance of the generated model.

The result is a very easy-to-use library that, with just four lines of code, automates the data preprocessing pipeline, selects the most promising hyperparameters, trains the best model and validates it. The library provides default values for all hyperparameters used, while being flexible enough to let the user overwrite these values if desired.

RESUMEN

Las redes neuronales artificiales son modelos de computación muy potentes, pero requieren del ajuste de numerosos hiperparámetros para lograr un buen rendimiento. Este proceso de ajuste es muy costoso y complejo y es conocido como AutoML. Actualmente, no existe ninguna librería de AutoML para Keras orientada a problemas de clasificación genéricos.

Se ha diseñado una heurística para ir fijando los distintos hiperparámetros iterativamente, tratando de emplear el menor coste computacional posible para la selección de estos. Esta heurística se ha implementado en una librería de Python, y se ha validado la librería con conjuntos de datos reales. Se ha analizado la evolución del rendimiento de los modelos generados al incrementar los recursos computacionales utilizados por la librería, comparando el rendimiento final de estos modelos con el rendimiento publicado en la literatura. Este proceso ha permitido llegar a un compromiso entre los recursos computacionales que requiere la librería, y la calidad del modelo final.

El resultado final es una librería muy sencilla de usar, que, con tan solo cuatro líneas de código, permite automatizar todo el preprocesamiento de los datos, seleccionar los hiperparámetros más prometedores, entrenar el mejor modelo y validarlo. La librería proporciona valores por defecto para todos los hiperparámetros que se usa, pero ofrece la posibilidad de permitir que el usuario sobrescriba esos valores por defecto.

ÍNDICE

1 INTRODUCCIÓN	1
1.1 INTRODUCCIÓN A LA INTELIGENCIA ARTIFICIAL	1
1.2 INTRODUCCIÓN AL MACHINE LEARNING.....	1
1.2.1 Clasificación de los tipos de Machine Learning.....	2
1.2.2 Introducción al Problema de Clasificación en Machine Learning.....	3
1.2.3 Redes neuronales y Deep Learning.....	4
1.2.4 Parámetros e hiperparámetros	7
1.2.5 Keras	9
1.2.6 Keras Tuner (búsqueda de hiperparametros)	11
1.2.7 Introducción a AutoML	13
1.2.8 Objetivos.....	14
2 ENTORNO DE DESARROLLO.....	15
2.1 PYTHON Y LIBRERÍAS	15
2.2 ENTORNO.....	16
3 PROPUESTA TÉCNICA Y DESARROLLO	18
3.1 ESTRUCTURA DEL FLUJO DE LA LIBRERÍA.....	18
3.2 PREPROCESAMIENTO	19
3.3 INICIALIZACIÓN CLASE AUTOMLCLASSIFIER	22
3.4 AJUSTE AUTOMÁTICO (AUTOTUNE).....	29
3.5 ENTRENAMIENTO DEL MODELO FINAL, EVALUACIÓN Y SELECCIÓN DE MEJORES HIPERPARÁMETROS	
32	
3.6 VALIDACIÓN.....	33
4 RESULTADOS Y DISCUSIÓN	35
4.1 EJEMPLO DE USO.....	35
4.2 RESULTADOS SOBRE LOS DATASETS DE PRUEBA	36
4.2.1 Ajuste de los hiperparámetros de la librería de autotune.....	36
4.2.2 Evaluación del rendimiento de la librería	42
5 CONCLUSIONES Y TRABAJO FUTURO.....	45
6 BIBLIOGRAFÍA.....	47

ÍNDICE DE FIGURAS

FIGURA 1. UN CONJUNTO DE DATOS ETIQUETADOS DE ENTRENAMIENTO PARA UN EJEMPLO DE APRENDIZAJE AUTOMÁTICO SUPERVISADO DE CLASIFICACIÓN DE SPAM [3].	2
FIGURA 2. NEURONA ARTIFICIAL BASE DE UNA ARQUITECTURA PERCEPTRÓN [3].....	5
FIGURA 3. ARQUITECTURA DE PERCEPTRÓN MULTICAPA CON DOS ENTRADAS, UNA CAPA OCULTA DE CUATRO NEURONAS Y UNA CAPA DE SALIDA DE TRES NEURONAS [3].	7
FIGURA 4. EJEMPLO DE RED NEURONAL WIDE&DEEP COMPLEJA USANDO API FUNCIONAL EN LA QUE EL FLUJO NO ES LINEAL.	10
FIGURA 5. EJEMPLO DE ARQUITECTURA DE RED NEURONAL CON API SECUENCIAL CON UNA CAPA DE ENTRADA DE DOS NEURONAS, DOS CAPAS OCULTAS DE TRES NEURONAS Y UNA CAPA DE SALIDA CON UNA NEURONA.....	11
FIGURA 6. ESTRUCTURA DEL FLUJO DE USO DE LA LIBRERÍA. EL COLOR NARANJA REPRESENTA PROCESOS, MIENTRAS QUE EL COLOR VERDE REPRESENTA ENTRADAS DE DATOS A UN PROCESO.	18
FIGURA 7. DIAGRAMA DE FLUJO DEL PROCESO DE PREPROCESAMIENTO DE UN DATASET.	19
FIGURA 8. EJEMPLO DE CODIFICACIÓN ONE-HOT ENCODING PARA CLASIFICACIÓN MULTICLASE.	21
FIGURA 9. DIAGRAMA DE FLUJO DEL PROCESO DE INICIALIZACIÓN DE LA CLASE <i>AUTOMLCLASSIFIER</i>	22
FIGURA 10. DIAGRAMA DE FLUJO DEL PROCESO DE PROCESO DE AJUSTE AUTOMÁTICO DE LOS HIPERPARÁMETROS DE UN MODELO DE RED NEURONAL. EL COLOR AZUL EN EL DIAGRAMA REPRESENTA UNA TOMA DE DECISIONES.	31
FIGURA 11. EJEMPLO DE USO DE LA CLASE <i>AUTOMLCLASSIFIER</i> PARA AUTOTUNING, ENTRENAMIENTO, EVALUACIÓN Y OBTENCIÓN DEL MODELO RESULTANTE.....	35
FIGURA 12. RESULTADO DE LA EJECUCIÓN DEL CÓDIGO PARA EL <i>DATASET</i> MUSHROOM.....	36
FIGURA 13. GRÁFICA DE PÉRDIDA EN FUNCIÓN DEL NÚMERO DE ÉPOCAS DE BÚSQUEDA.	40
FIGURA 14. GRÁFICA DE PRECISIÓN EN FUNCIÓN DEL NÚMERO DE ÉPOCAS DE BÚSQUEDA.	41
FIGURA 15. GRÁFICA DE PÉRDIDA EN FUNCIÓN DEL NÚMERO DE TRIALS.	41
FIGURA 16. GRÁFICA DE PRECISIÓN EN FUNCIÓN DEL NÚMERO DE TRIALS.....	42

ÍNDICE DE TABLAS

TABLA 1. RESUMEN DE VALORES DE HIPERPARÁMETROS ASOCIADOS A LA DIVISIÓN DEL CONJUNTO DE DATOS EN ENTRENAMIENTO, TEST Y VALIDACIÓN.....	23
TABLA 2. RESUMEN DE VALORES DE HIPERPARÁMETROS ASOCIADOS AL ENTRENAMIENTO DEL MODELO..	25
TABLA 3. RESUMEN DE VALORES DE HIPERPARÁMETROS ASOCIADOS A LA ARQUITECTURA DEL MODELO. D ES EL NÚMERO DE CARACTERÍSTICAS Y C ES EL NÚMERO DE CLASES OBJETIVO.	27
TABLA 4. RESUMEN DE VALORES DE HIPERPARÁMETROS ASOCIADOS AL OPTIMIZADOR.....	28
TABLA 5. RESUMEN DE VALORES DE HIPERPARÁMETROS ASOCIADOS AL TUNER.	29
TABLA 6. RESUMEN DE CARACTERÍSTICAS DE LOS DISTINTOS DATASETS UTILIZADOS PARA LA VALIDACIÓN DE LA LIBRERÍA.	34
TABLA 7. INFORMACIÓN OFRECIDA POR UCIMLREPO SOBRE PRECISIÓN MÁXIMA ALCANZABLE POR UN MODELO DE RED NEURONAL DE CLASIFICACIÓN PARA CADA DATASET UTILIZADO [23-27].	34
TABLA 8. MEJORES RESULTADOS DE LA LIBRERÍA PARA CADA DATASET DE PRUEBA.....	44
TABLA 9. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET BANK MARKETING FIJANDO EL NÚMERO MÁXIMO DE TRIALS POR ETAPA DE BÚSQUEDA.....	49
TABLA 10. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET BANK MARKETING FIJANDO EL NÚMERO DE ÉPOCAS DE ENTRENAMIENTO POR TRIAL.	49
TABLA 11. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET BREAST CANCER WISCONSIN FIJANDO EL NÚMERO MÁXIMO DE TRIALS POR ETAPA DE BÚSQUEDA.	50
TABLA 12. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET BREAST CANCER WISCONSIN FIJANDO EL NÚMERO DE ÉPOCAS DE ENTRENAMIENTO POR TRIAL.	50
TABLA 13. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET CAR EVALUATION FIJANDO EL NÚMERO MÁXIMO DE TRIALS POR ETAPA DE BÚSQUEDA.....	51
TABLA 14. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET CAR EVALUATION FIJANDO EL NÚMERO DE ÉPOCAS DE ENTRENAMIENTO POR TRIAL.	51
TABLA 15. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET CENSUS INCOME FIJANDO EL NÚMERO MÁXIMO DE TRIALS POR ETAPA DE BÚSQUEDA.....	51
TABLA 16. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET CENSUS INCOME FIJANDO EL NÚMERO DE ÉPOCAS DE ENTRENAMIENTO POR TRIAL.	52
TABLA 17. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET DEFAULT OF CREDIT CARD CLIENTS FIJANDO EL NÚMERO MÁXIMO DE TRIALS POR ETAPA DE BÚSQUEDA.	52
TABLA 18. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET DEFAULT OF CREDIT CARD CLIENTS FIJANDO EL NÚMERO DE ÉPOCAS DE ENTRENAMIENTO POR TRIAL.	53
TABLA 19. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET DRY BEAN FIJANDO EL NÚMERO MÁXIMO DE TRIALS POR ETAPA DE BÚSQUEDA.	53
TABLA 20. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET DRY BEAN FIJANDO EL NÚMERO DE ÉPOCAS DE ENTRENAMIENTO POR TRIAL.	53

TABLA 21. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET HEART DISEASE FIJANDO EL NÚMERO MÁXIMO DE TRIALS POR ETAPA DE BÚSQUEDA.	54
TABLA 22. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET HEART DISEASE FIJANDO EL NÚMERO DE ÉPOCAS DE ENTRENAMIENTO POR TRIAL.	54
TABLA 23. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET MAGIC GAMMA TELESCOPE FIJANDO EL NÚMERO MÁXIMO DE TRIALS POR ETAPA DE BÚSQUEDA.	54
TABLA 24. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET MAGIC GAMMA TELESCOPE FIJANDO EL NÚMERO DE ÉPOCAS DE ENTRENAMIENTO POR TRIAL.	55
TABLA 25. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET MUSHROOM FIJANDO EL NÚMERO MÁXIMO DE TRIALS POR ETAPA DE BÚSQUEDA.	55
TABLA 26. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET MUSHROOM FIJANDO EL NÚMERO DE ÉPOCAS DE ENTRENAMIENTO POR TRIAL.	55
TABLA 27. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET SPAMBASE FIJANDO EL NÚMERO MÁXIMO DE TRIALS POR ETAPA DE BÚSQUEDA.	56
TABLA 28. RESULTADOS DE LA LIBRERÍA SOBRE EL DATASET SPAMBASE FIJANDO EL NÚMERO DE ÉPOCAS DE ENTRENAMIENTO POR TRIAL.	56

1 INTRODUCCIÓN

1.1 Introducción a la Inteligencia Artificial

La Inteligencia Artificial (IA) se puede entender como la capacidad de una máquina de realizar tareas que durante toda la historia han sido asociadas a una inteligencia exclusivamente humana. De esta manera, se entiende que la máquina es capaz de llevar a cabo tareas tales como el razonamiento, la percepción, la comprensión del lenguaje y la toma de decisiones [1].

A nivel más técnico, la IA hoy en día se compone de una mezcla entre la representación del conocimiento, lógica formal, búsqueda heurística, procesamiento de lenguaje natural, visión por computador y aprendizaje. La relevancia de cada una de estas disciplinas varía según la finalidad del sistema de IA. Inicialmente, estas áreas fueron estudiadas de forma individual, aunque hoy en día se usan e integran en sistemas muy complejos e interconectados [1].

1.2 Introducción al Machine Learning

El aprendizaje automático (Machine Learning o ML, en inglés) es un subcampo dentro de la inteligencia artificial que trata de desarrollar sistemas informáticos que aprenden automáticamente de los datos sin necesidad de ser programados explícitamente para ello. Es decir, permite mejorar automáticamente el rendimiento del programa informático [2]. En el pasado, el paradigma dominante de la inteligencia artificial (sistemas expertos) consistía en que los científicos programaban manualmente las reglas que definían como debían comportarse los sistemas expertos. Estas reglas se obtenían a partir de seres humanos que sabían cómo resolver el problema en cuestión. Con esta aproximación sólo es posible resolver un problema que ya se conoce cómo se puede resolver, y que dicho conocimiento es explícito y no tácito. Por otro lado, la adquisición del conocimiento experto resultó ser un cuello de botella y requerir una gran cantidad de recursos. Con la aproximación actual, el aprendizaje automático, se analizan automáticamente grandes volúmenes de datos para inferir patrones y comportamientos.

El primer sistema que se popularizó enormemente que utilizaba aprendizaje automático fue el filtro de spam del correo electrónico, pero, poco a poco, el aprendizaje automático se ha consolidado como el núcleo de una gran variedad de aplicaciones modernas que utilizan sistemas de IA, llegando a impulsar tecnologías como la traducción automática, el reconocimiento de imágenes, la recomendación de productos, los vehículos autónomos y el procesamiento de lenguaje natural [3].

Con el ejemplo previo del filtro de spam del correo electrónico se observan claramente los beneficios de esta técnica: en lugar de detectar un tipo de estructura de email que contiene spam y posteriormente crear una regla a mano que lo identifique y lo trate, se puede programar que el filtro detecte automáticamente las estructuras y contenidos de spam y los trate debidamente, sin tener que ir manualmente uno a uno entre todos los tipos de correos y spam [3].

1.2.1 Clasificación de los tipos de Machine Learning

Existen diferentes tipos de clasificación del aprendizaje automático en función de distintas características del sistema. Según la supervisión durante el entrenamiento, puede clasificarse en:

- *Aprendizaje supervisado*: En este tipo de aprendizaje automático, el conjunto de datos de entrenamiento incluye las soluciones que se desean, es decir, aprende a través de datos etiquetados previamente con el objetivo de conseguir predecir un nuevo dato entrante que no esté etiquetado (ver Figura 1).

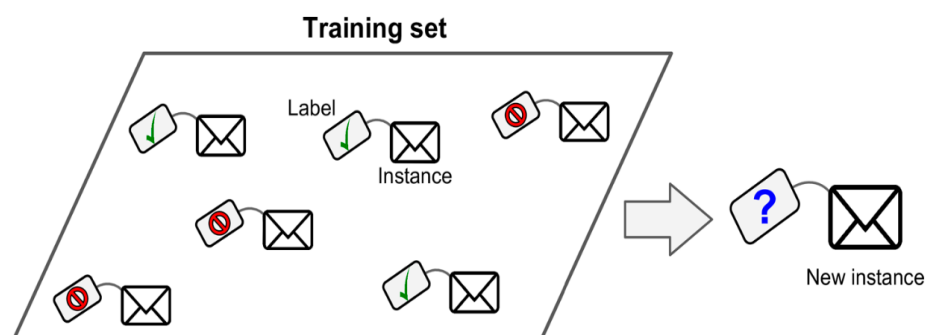


Figura 1. Un conjunto de datos etiquetados de entrenamiento para un ejemplo de aprendizaje automático supervisado de clasificación de spam [3].

En muchos casos, se etiquetan los datos manualmente para poder pasárselos al modelo durante el entrenamiento y que aprenda por sí solo. Se llama modelo a la representación computacional que aprende patrones o estructuras de los datos de entrenamiento con el objetivo de hacer predicciones sobre un nuevo dato de entrada determinado. Un ejemplo típico de entrenamiento supervisado es el filtro de spam del correo electrónico ya mencionado anteriormente. En este ejemplo, al modelo que se usa para la detección de spam se le pasan los correos electrónicos etiquetados verificando si el correo era spam o no, con el objetivo de que el modelo consiga encontrar patrones y analizarlos automáticamente [3].

- *Aprendizaje no supervisado*: El aprendizaje no supervisado consiste en entrenar un modelo de aprendizaje automático a través de un conjunto de datos de entrenamiento que no están etiquetados. Es decir, durante el entrenamiento, el modelo no dispone de información sobre la respuesta o salida correcta asociada a cada instancia del conjunto de datos.
- *Aprendizaje semi-supervisado*: En este enfoque, el modelo se entrena con un conjunto de datos que contiene tanto datos etiquetados como no etiquetados. Dado que etiquetar los datos puede llegar a ser un proceso difícil y consumir bastante tiempo, se opta por aprovechar ambos tipos de datos para optimizar el modelo. Se puede entender, en la mayoría de los casos, como una mezcla entre el aprendizaje no supervisado y aprendizaje supervisado.
- *Aprendizaje auto-supervisado*: Este tipo de aprendizaje se caracteriza por utilizar como entrada al entrenamiento del modelo un conjunto de datos sin etiquetar y posteriormente generar las etiquetas a través de los propios datos. Esta técnica, al igual que ocurría con el aprendizaje semi-supervisado, es un enfoque intermedio entre el aprendizaje supervisado y aprendizaje no supervisado.

1.2.2 Introducción al Problema de Clasificación en Machine Learning

La Inteligencia Artificial y el Machine Learning han transformado por completo la manera en que se abordan el análisis y la resolución de problemas en diversas disciplinas. A grandes rasgos, los problemas que el Machine Learning puede abordar son:

regresión, clasificación, agrupamiento o clustering, detección de anomalías y reducción de la dimensionalidad. De entre todos ellos, este estudio se centrará en los problemas de clasificación, dado que todos ellos comparten una misma estructura de arquitectura de red neuronal genérica, por lo que se pueden sacar conclusiones válidas al comparar cómo se comportan estas arquitecturas para distintos conjuntos de datos del mismo tipo de problema.

El objetivo de un problema de clasificación es asignar una clase o categoría a cada nueva instancia, para lo cual el modelo se basa en su previo aprendizaje de los datos etiquetados de entrenamiento. Es decir, el modelo analiza las características del nuevo dato de entrada con el objetivo de deducir a qué clase pertenece. Las clases son las etiquetas o categorías a las que un modelo puede asignar una instancia y son aprendidas durante el entrenamiento, por lo que el modelo no podrá asignar una instancia a una clase que no existía dentro de su conjunto de datos de entrenamiento; para el modelo esa clase sería inexistente, al no haberla aprendido en su entrenamiento. Formalmente, la tarea de clasificación implica encontrar una función $f: X \rightarrow Y$, donde X es el espacio de las características de entrada e Y es el conjunto finito de clases posibles. Por ejemplo, en un problema de clasificación binaria, $Y = \{0,1\}$; mientras que en clasificación multiclase, $Y = \{0,1,2, \dots, k-1\}$, donde k es el número de clases [2].

Como se puede apreciar dada la naturaleza del problema, la clasificación se basa en el aprendizaje automático supervisado, dado que necesita conocer las etiquetas de los datos de entrenamiento para poder aprender las clases existentes y analizar qué patrones determinan a qué clase pertenece una instancia determinada. Por tanto, un ejemplo ilustrativo de clasificación es el filtro de spam comentado con anterioridad.

1.2.3 Redes neuronales y Deep Learning

Las redes neuronales son la técnica de aprendizaje automático más popular en la actualidad. La estructura de una red neuronal se base en un conjunto de unidades llamadas neuronas artificiales, organizadas en capas interconectadas: una capa de entrada, una o varias capas ocultas y una capa de salida. En esencia, cada una de las neuronas de la red realiza una operación matemática muy sencilla, pero en su conjunto se pueden modelar funciones altamente no lineales. Para entrar en detalle, se explicará el funcionamiento de

un tipo de arquitectura de redes neuronales comúnmente usada, denominada *perceptrón*, la cual se basa en una neurona artificial llamada TLU (ver Figura 2).

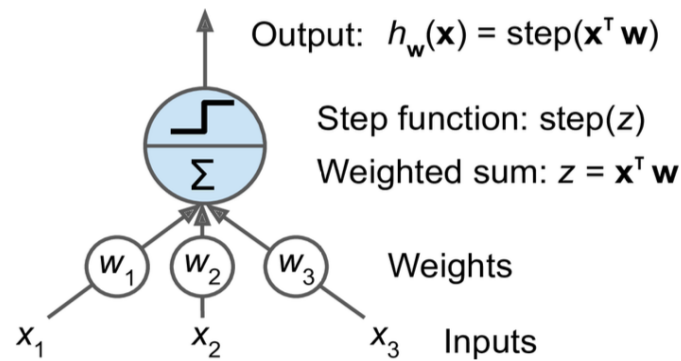


Figura 2. Neurona artificial base de una arquitectura perceptrón [3].

La neurona tiene un funcionamiento muy sencillo. En este caso, la TLU (Linear Threshold Unit) calcula una función lineal de sus entradas: $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \mathbf{w}^T \mathbf{x} + b$. A cada entrada de una neurona, independientemente de la capa en la que se encuentre, se le asocia un peso. Cuando una neurona recibe varias entradas, cada una se multiplica por su peso correspondiente ($w_1, w_2, \dots, w_n$) antes de ser sumadas. Los pesos permiten determinar la relevancia de cada entrada y capturar patrones en los datos. Por último, aplica una función escalón o similar al resultado. Este resultado puede ser la entrada a otra neurona de una capa oculta o de una capa de salida o simplemente ser el resultado final [3]. Si se juntan varias de estas neuronas en una estructura de capas se obtiene lo que se conoce como un perceptrón multicapa (ver Figura 3) [3].

Las neuronas que en la imagen se distinguen con un I (ver Figura 3), son el bias o neurona de sesgo, la cual actúa como un valor que permite que la función de activación se desplace, permitiendo aprender patrones más complejos. Sin las neuronas de bias, la frontera definida en el espacio de características por un perceptrón tendría que pasar siempre por el origen de coordenadas.

Cuando las redes neuronales tienen dos o más capas ocultas, se les suele denominar redes profundas (*Deep Learning*). Este tipo de arquitectura de red ha permitido avances extraordinarios en tareas como visión por computador, procesamiento del lenguaje natural o generación de contenidos sintéticos [4]. A diferencia de otros algoritmos utilizados en

Inteligencia Artificial que dependen de una ingeniería manual de características intensiva, el *Deep Learning* tiene la capacidad de aprender representaciones jerárquicas a partir de los datos automáticamente. Es decir, en los métodos tradicionales, los expertos debían estudiar las características del problema que se quería resolver, de manera que se decidía qué atributos y características serían útiles para la tarea en cuestión. Por ejemplo: sin *Deep Learning*, un experto podría estudiar un problema relacionado con imágenes, tomando como conclusión que se deben tener en cuenta los bordes, texturas o formas mediante filtros, todo ello de forma manual. En cambio, con *Deep Learning*, la red neuronal recibe los píxeles de las imágenes directamente y esta aprende por sí sola cuáles son los patrones útiles. La red podría establecer, por ejemplo, que las primeras capas deberían aprender a detectar bordes, las siguientes determinar formas simples como esquinas o contornos y las últimas, formas más complejas como caras o letras.

Aunque todo indica a que usar una red neuronal profunda, debido a sus grandes ventajas respecto a la capacidad de aprender patrones y automatizar el proceso de estudio del conjunto de datos, es lo que mejores resultados va a obtener, esta también tiene ciertas limitaciones o desventajas. Entre ellas, la más destacable es la necesidad de entrenamiento con una gran cantidad de datos. En la mayoría de los casos, es necesario entrenar el modelo con millones de datos etiquetados para ver una mejoría notable en la predicción de resultados [5]. Esto supone un gasto en tiempo enorme, explicado por dos motivos. En primer lugar, el tiempo costoso de etiquetado de los datos de entrenamiento, que son esenciales para el entrenamiento del modelo, como ya se ha comentado anteriormente. En segundo lugar, el tiempo de entrenamiento en sí mismo, el cual puede llegar a tardar varias semanas o meses, dependiendo de la cantidad de datos. Además, cabe resaltar que lo más probable es que se necesiten máquinas muy potentes que soporten toda esta carga de datos intentando reducir el tiempo de cómputo al mínimo posible.

Otro gran inconveniente asociado a los modelos de redes neuronales profundas es el hecho de que, al tener una estructura muy compleja, con muchas neuronas por capa y muchas capas, da lugar a un número de parámetros entrenables inmenso. Dado que esto es así, hay tendencia a que ocurra lo que se conoce como sobreajuste, que significa que el modelo aprende de memoria el conjunto de datos de entrenamiento, siendo muy capaz de replicar los resultados que ya ha visto o experimentado durante el entrenamiento, pero

con una falta de capacidad para generalizar ante nuevas instancias entrantes, es decir, que aprenda a dar un resultado correcto ante una nueva instancia de dato entrante, que en definitiva es el objetivo del modelo. Por ello, muchas veces se opta por técnicas conocidas como regularización, que permiten obligar al modelo a generalizar en caso de que se detecten casos de sobreajuste u *overfitting* [6].

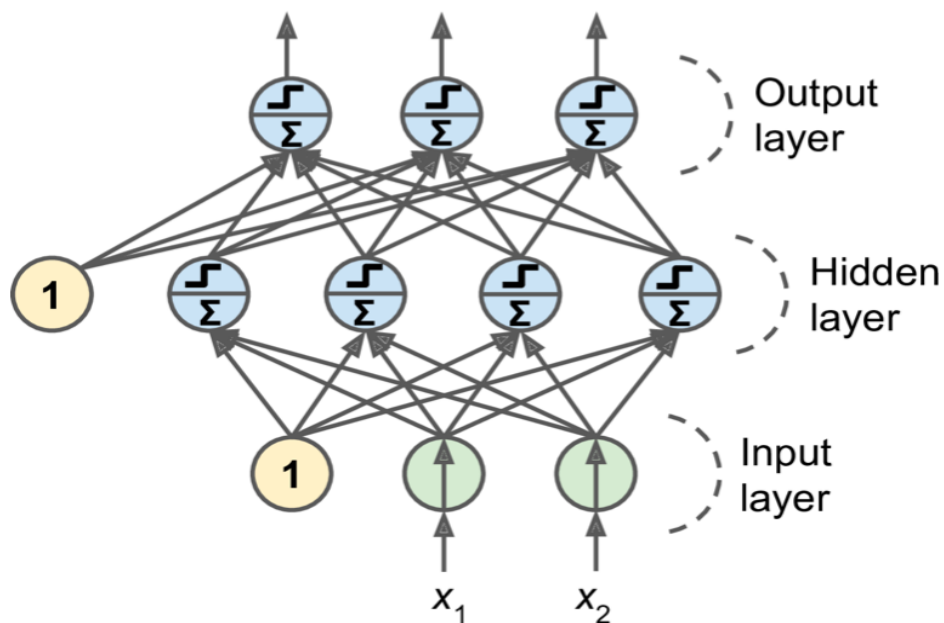


Figura 3. Arquitectura de perceptrón multicapa con dos entradas, una capa oculta de cuatro neuronas y una capa de salida de tres neuronas [3].

1.2.4 Parámetros e hiperparámetros

Los parámetros de un modelo se definen como todo el conjunto de valores internos que son elegidos y modificados a través del entrenamiento. En una red neuronal, estos son los pesos de las neuronas y los *biases* o sesgos de las conexiones entre neuronas. El propósito del entrenamiento de una red neuronal es encontrar unos valores que consigan que la red realice la tarea deseada. Para ello, se utiliza el algoritmo de *backpropagation* o retropropagación. El algoritmo consta de dos pasos: uno hacia adelante y otro hacia atrás. Primero, se hace pasar una entrada por toda la red neuronal, con los pesos y *biases* actuales, hasta llegar a la capa de salida (paso hacia adelante),

donde se calcula el error cometido, expresado como la comparación entre el resultado obtenido y el real. Una vez se tiene el error que se ha producido en la salida, se inicia un proceso inverso (paso hacia atrás), por el cual los pesos y *biases* se modifican siguiendo una regla en cadena a partir del uso de descenso de gradiente sobre la función de pérdida, con el objetivo de indicar la dirección y magnitud del cambio en los parámetros. Repitiendo este proceso se consigue que, poco a poco con todos los datos de entrenamiento, la red aprenda a ajustar los parámetros y mejorar su rendimiento [7].

Los hiperparámetros, en cambio, son las propiedades del modelo que son elegidas antes del entrenamiento, ya que definen como el modelo va a comportarse a la hora de aprender. Entre los principales hiperparámetros destacan:

- *Número de capas ocultas*: Determina la profundidad de la red neuronal. Los valores típicos suelen estar entre 1 y 10 [6].
- *Número de neuronas por capa oculta*: Controla el tamaño de cada capa oculta. Los valores típicos se sitúan entre las decenas y varios cientos [6].
- *Función de activación*: Transforma la salida de una neurona en la entrada de la siguiente neurona introduciendo no linealidades permitiendo aprender patrones más complejos entre entradas y salidas [8].
- *Tasa de aprendizaje*: Controla la magnitud de la variación de las correcciones de los parámetros en el algoritmo de retropropagación. Es un hiperparámetro crítico para el correcto funcionamiento de la red neuronal.
- *Optimizador*: Se encarga de calcular el descenso de gradiente del entrenamiento de la red neuronal. Se apoya en la tasa de aprendizaje para llevar a cabo el cálculo. También puede incluir técnicas de regularización (como la aplicación de algún límite de la magnitud del gradiente), que ayudan a evitar el sobreajuste del modelo [9]. También existen varios hiperparámetros internos dentro del optimizador, en función del tipo de optimizador. Dos de los más habituales son:
 - *Momentum o momento*: Habilita la acumulación de actualizaciones en la dirección de máxima pendiente en el descenso de gradiente, permitiendo acelerar la convergencia del algoritmo [10].
 - *Nesterov*: Calcula los gradientes anticipando o previendo los futuros valores de estos, permitiendo acelerar la convergencia del algoritmo [11].

- *Acorte (clip) de gradientes*: Es una técnica que limita la magnitud de los gradientes para evitar inestabilidades provocadas por explosiones de gradiente. Una técnica de clip de gradientes es *Clipnorm*, que normaliza el vector de gradiente si su norma supera un cierto valor establecido [12].
- *Tamaño de batch o lote*: Número de instancias de datos que conforman un lote. Los parámetros se actualizan por cada lote, por lo que este valor influye en la aplicación del algoritmo de retropropagación que se aplica a la red neuronal. Si el tamaño es pequeño existirán más actualizaciones de los parámetros de la red neuronal que si el tamaño es grande, pero cuanto más pequeño sea este parámetro se realizará en base a un menor número de datos y puede haber una mayor variabilidad en la dirección de estas actualizaciones.
- *Número de épocas*: Dado que una época se define como una pasada completa por todo el conjunto de datos de entrenamiento, el número de épocas indica cuántas veces recorrer el conjunto entero. Existe una relación directa entre una época y el tamaño del *batch*, por ejemplo: si el conjunto de datos tiene 1000 instancias y el tamaño del *batch* es de 100 instancias, entonces una época son 10 actualizaciones de pesos, ya que serán procesados 10 *batches*.

Especialmente en una red neuronal profunda, hay una gran cantidad de hiperparámetros sobre los que el analista debe decidir, antes de proceder a la estimación de los parámetros de la red. No existen fórmulas bien definidas o criterios ampliamente aplicables para tomar decisiones sobre estos hiperparámetros, requiriendo su ajuste un proceso de prueba y error informado por la experiencia previa del analista. Esto es un proceso complejo y tedioso, del cual a menudo depende el éxito de la tarea que se está abordando.

1.2.5 Keras

Para la realización del modelo de red neuronal, se necesita de una biblioteca de código que permita su creación. Hay varias alternativas, como, por ejemplo: PyTorch, TensorFlow, Haiku o Keras, entre muchas otras. En este trabajo se ha decidido utilizar Keras, debido a que tiene una curva de aprendizaje mucho más suave que otras alternativas como Pytorch o TensorFlow, por lo que es una herramienta ideal para

desarrollar un proyecto como este. Además, ofrece una interfaz de alto nivel que permite la utilización rápida y sencilla de redes neuronales desde el primer momento.

Keras es una biblioteca de alto nivel en Python creada por Francis Chollet en 2015. Es tan usada, que hoy en día forma parte de TensorFlow, la cual es una librería más amplia de *machine learning* y de más bajo nivel. Este hecho permite una mayor cohesión entre el código creado en Keras y su utilidad para otras aplicaciones montadas sobre TensorFlow. Keras tiene una API (*Application Programming Interface*) muy sencilla que permite construir redes neuronales conectando las capas de manera clara y sencilla. Hay dos APIs: secuencial (*Sequential*) y funcional (*Functional API*). La API secuencial es más simple de leer y aprender, ya que se fundamenta en tener una configuración compacta y lineal de las capas, donde las capas se apilan unas encima de otras formando la red neuronal. Es perfecta para redes neuronales con un único flujo de datos desde la entrada de la red hasta la salida. En cambio, la API funcional puede llegar a ser más compleja de entender y utilizar, pero con la ventaja de poder llegar a crear redes neuronales bastante más complejas con bifurcaciones, ramificaciones y varias entradas y salidas (ver Figura 4). Es una alternativa muy interesante si se requiere una arquitectura avanzada, como ResNet o redes Wide&Deep.

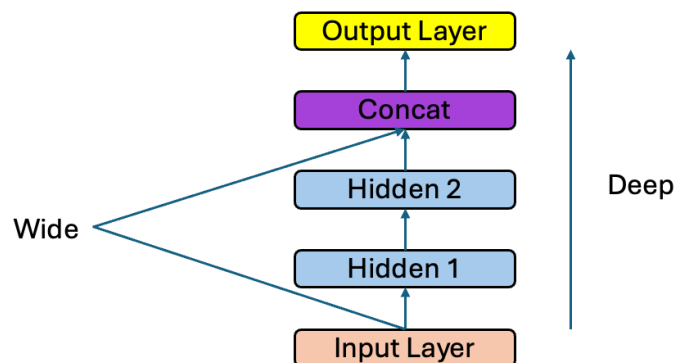


Figura 4. Ejemplo de red neuronal Wide&Deep compleja usando API funcional en la que el flujo no es lineal.

En el caso de este trabajo, se ha escogido la API secuencial, ya que no se va a llegar a trabajar con conjuntos de datos ni problemas que requiera de una arquitectura de red neuronal tan compleja. El objetivo es dar servicio a problemas básicos de

clasificación, donde hay un flujo lineal de la información que pasa por la red neuronal, por lo que no existe complejidad en el diseño de las capas, ya que se apilan en el orden que se muestra en la imagen: capa de entrada, capas intermedias u ocultas y capa de salida (ver Figura 5).

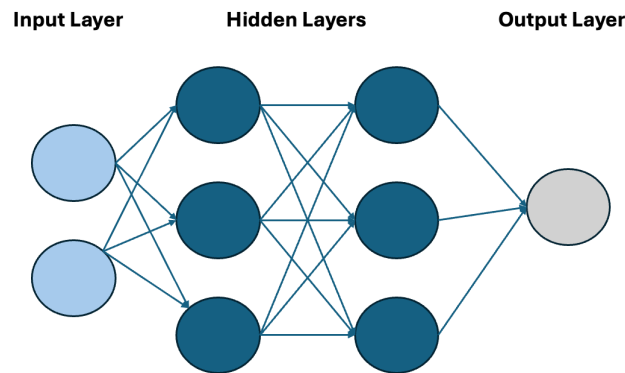


Figura 5. Ejemplo de arquitectura de red neuronal con API secuencial con una capa de entrada de dos neuronas, dos capas ocultas de tres neuronas y una capa de salida con una neurona.

1.2.6 Keras Tuner (búsqueda de hiperparámetros)

La búsqueda de hiperparámetros es crucial para encontrar el rendimiento óptimo del modelo, ya que antes de entrenarlo, se necesita tener definida una estructura que permita que los algoritmos de entrenamiento converjan y que consigan que el modelo aprenda y generalice. Además, se debe evitar el sobreajuste y el modelo debe adaptarse correctamente a la naturaleza de los datos para sacar el mayor potencial. Antes de entrenar un modelo, por tanto, se busca probar muchas combinaciones de hiperparámetros para ver cuáles son las mejores, con el objetivo de entrenar el modelo con los mejores valores posibles. Para ello, existen varias librerías que permiten automatizar esta búsqueda.

Existen librerías que son más generalistas, ya que aplican a diferentes plataformas de creación de código de modelos de Inteligencia Artificial: Optuna, Ray Tune, Hyperopt, Nevergrad. En su mayoría, son compatibles con TensorFlow, PyTorch y scikit-learn, los cuales son frameworks de desarrollo de código ampliamente conocidos y usados. Otras librerías son más específicas de un entorno en particular: Keras Tuner

(para Keras) o GridSearchCV y RandomizedSearchCV (para scikit-learn). Existen también otras alternativas más académicas: SMAC (usada en Auto-sklearn) y BOHB.

Keras Tuner es una buena alternativa para realizar la búsqueda de hiperparámetros de un modelo de Keras, debido a que es la única herramienta diseñada para trabajar específicamente sobre Keras. El resto de las librerías, aunque parecidas en algunos puntos, no dan soporte total sobre el código de Keras, ya que están diseñadas para trabajar con TensorFlow, no con Keras en específico. Otras librerías, como SMAC, que se usa con el mismo fin que tiene este trabajo (AutoML), está diseñada para utilizarse sobre sk-learn, por lo que tampoco es lo que se busca.

Con Keras Tuner, el usuario define un espacio de búsqueda de hiperparámetros, dando la posibilidad de escoger los valores entre los que realizará la búsqueda, ya sean binarios, enteros, enumerados o con decimales. Con ese espacio de búsqueda y dado un algoritmo de búsqueda, Keras Tuner itera sobre una función de exploración que entrena un modelo con diferentes valores de hiperparámetros del espacio de búsqueda y busca los mejores resultados posibles. Es decir, ejecuta la misma función muchas veces, cambiando el valor de los hiperparámetros en cada vuelta (cada vuelta se conoce como *trial*). Finalmente, devuelve los resultados de todos los *trials*, siendo posible escoger los hiperparámetros del mejor *trial* de manera muy sencilla [13].

Es importante resaltar la diferencia entre el concepto de *tuner* y AutoML. En el primero, es necesario especificar los rangos de búsqueda de los hiperparámetros, por lo que se necesita tener un conocimiento avanzado sobre el funcionamiento interno de las redes neuronales y sobre el problema que se desea resolver. Por ejemplo, el usuario puede especificar que se van a emplear entre 1 o 30 capas ocultas en su modelo, lo que da lugar a unas variaciones muy grandes en los resultados obtenidos. Sin el conocimiento técnico necesario, el usuario se podría ver incapaz de restringir la búsqueda para obtener unos rangos razonables que provoquen que el modelo pueda converger correctamente en un tiempo determinado. En el segundo, en cambio, se pretende abstraer este conocimiento para que el usuario, sin ninguna noción sobre el tema, pueda utilizar la librería y obtener unos resultados aceptables.

1.2.7 Introducción a AutoML

AutoML es una técnica cuyo objetivo final es permitir al usuario conseguir una arquitectura de modelo de IA decente automatizando todo el proceso. Hay diferentes aproximaciones, ya que algunas se centran en automatizar solamente un paso específico de todo el proceso, como podría ser el caso de Keras Tuner, pero el problema de la necesidad de conocimiento sobre arquitecturas persiste, debido a que se sigue necesitando establecer los rangos de búsqueda para que el problema sea realizable (ver Ajuste automático (autotune)). Otras alternativas, como AutoKeras o Auto-sklearn buscan que el usuario tenga la posibilidad de conseguir un buen modelo pasando solamente el conjunto de datos con el que se quiere entrenar el modelo. Por tanto, ofrecen un *pipeline* opaco para automatizar todo el proceso de principio a fin una vez se obtiene el conjunto de datos, es decir, se encargan de la limpieza de datos, selección de características, elección del modelo, ajuste de parámetros e hiperparámetros, combinación de modelos, monitorización, despliegue, etc. Estas herramientas mencionadas, por tanto, ocultan por completo el flujo interno de búsqueda, sin ofrecer la posibilidad al usuario, en caso de tener cierto conocimiento técnico, de poder elegir hiperparámetros fijos y probar distintas arquitecturas en base a ellos. Para el caso específico de AutoKeras, esta librería solo ofrece la resolución de ciertos problemas específicos: clasificación de imágenes y texto, así como regresión de imágenes y texto. Cabe resaltar que otro aspecto relevante de AutoKeras es el hecho de que realiza una búsqueda más en profundidad a través de una técnica conocida como NAS (Neural Architecture Search), explorando arquitecturas en paralelo, que, aunque provoque búsquedas más exhaustivas, necesita un mayor potencial de cálculo y procesamiento [14].

Por tanto, no existe actualmente ninguna librería aplicada a Keras que permita crear un modelo de red neuronal para clasificación genérica y que además el proceso de búsqueda sea personalizable a distintos niveles según el conocimiento técnico del usuario. En definitiva, el proyecto realizado en este trabajo se puede visualizar como una idea novedosa para cubrir las necesidades de profesionales del sector de la Inteligencia Artificial que no se encuentran satisfechos con las herramientas actuales de AutoML compatibles con Keras.

1.2.8 Objetivos

Como se ha comentado anteriormente, actualmente no existe una librería que ofrezca la posibilidad de encapsular el proceso completo de búsqueda de hiperparámetros de un modelo de Keras para un problema de clasificación general, pero que a la vez permita al usuario limitar esa búsqueda cuando lo considere oportuno. El objetivo de este trabajo es crear una librería en Python compatible con Keras, que permita cubrir esta carencia, específicamente para problemas de clasificación genéricos usando redes neuronales. Esta librería deberá requerir la menor capacidad de cómputo posible durante el proceso de búsqueda de hiperparámetros, detectando patrones temprano para evitar explorar combinaciones de hiperparámetros que no den buenos resultados. A la vez, la arquitectura final de red neuronal debería tener el mejor rendimiento posible. Para conseguir este objetivo, es necesario:

- Diseñar una estrategia de AutoML que emplee métodos heurísticos para ir fijando hiperparámetros sucesivamente sin requerir una gran cantidad de recursos computacionales para ello. Deberá ser posible generar un modelo pasando como entrada solamente los datos de entrenamiento. Aunque también se deberá permitir que los usuarios más avanzados puedan especificar rangos de búsqueda para los hiperparámetros.
- Realizar una implementación de dicha estrategia empleando Keras. La implementación deberá soportar *features* de entrada, tanto métricas como nominales, así como problemas de clasificación binarios y multiclase. No será necesario que se haya realizado un preprocesamiento previo de los datos por parte del usuario.
- Estudiar la evolución de los modelos resultantes de la librería en función del número de épocas entrenadas y número de *trials* escogidos para la cada búsqueda, con el fin de optimizar la implementación de la librería.
- Validar la implementación final de la librería de AutoML empleando varios conjuntos de datos públicos, tanto para verificar su correcto funcionamiento como para evaluar su rendimiento.

2 ENTORNO DE DESARROLLO

2.1 Python y librerías

El proyecto se ha realizado en Python, un lenguaje de programación que ha experimentado un gran auge en los últimos años debido a su filosofía de utilizar una sintaxis clara y sencilla, muy útil para trabajar con temas científicos complejos. Por ello, muchos científicos alrededor del mundo han desarrollado librerías que se centran en temas muy diversos, ofreciendo una gama amplia de utilidades para cualquier técnico que lo necesite. Además, consta de una comunidad activa que en muchos casos se agradece a la hora de buscar información sobre problemas encontrados y poder ver diferentes aproximaciones a una solución en común.

Las librerías utilizadas para desarrollar Inteligencia Artificial en Python en el proyecto han sido Scikit-learn, pandas, NumPy, TensorFlow (versión 2.15.0), Keras (versión 2.15.0), y Keras Tuner (versión 1.4.7). Scikit-learn es una potente librería con un ecosistema muy rico, que hoy en día incluye funciones que permiten construir un sistema de Inteligencia Artificial de principio a fin, es decir, desde el preprocesado del conjunto de datos que se recibe hasta la creación, utilización y validación del modelo. En este caso en concreto, se ha hecho uso de Scikit-learn para su faceta de preprocesado de datos, ya que incluye métodos que permiten imputar valores, escalar y codificar las variables, además de la funcionalidad de dividir un conjunto de datos para su uso como datos de entrenamiento y de test o validación. Por su parte, pandas y NumPy son la base de cualquier flujo de datos en Python, ya que pandas ofrece una buena manipulación de datos tabulares, incluyendo la creación y exploración de estructuras de datos propias muy intuitivas, como los pandas *dataframes*. En cambio, NumPy se focaliza más en estructuras numéricas de alto rendimiento y operaciones vectorizadas sobre arrays, esenciales para el cálculo científico y la preparación de tensores antes de pasarlos a un modelo.

Como ya se ha comentado anteriormente, TensorFlow es una librería de bajo nivel para trabajar con tensores. Debido a que puede llegar a ser complicado utilizarla, se han desarrollado otras librerías que abstraen el código de bajo nivel de TensorFlow y permiten ofrecer una capa de alto nivel para la creación de modelos, como es el caso de Keras. Keras se enfoca en redes neuronales y mantiene la posibilidad de llegar a hacer

modelos muy complejos en caso de ser necesario, pero con una interfaz más amigable que TensorFlow. En específico, se ha usado la librería *tensorflow.keras*, la cual ha sido creada recientemente y ofrece una interfaz de Keras que está construida directamente sobre TensorFlow, por lo que aumenta la compatibilidad e interoperabilidad del código. Finalmente, la última librería que cabe resaltar es Keras Tuner, la cual es muy útil para explorar diferentes arquitecturas de red neuronal y conseguir los mejores hiperparámetros para el posterior entrenamiento final del modelo. Como se puede observar, el entorno de programación se basa completamente en Python, con el objetivo de fomentar la contribución de otros científicos en el estudio de este tipo de proyectos y formar parte activamente del ecosistema de estudio de la IA.

2.2 Entorno

Se ha escogido utilizar el IDE (*Integrated Development Environment*) PyCharm Pro Edition, para el cual hace falta tener una licencia de pago. Es un editor especializado en Python, el cual permite el autocompletado inteligente y tiene incluido un buscador de variables y módulos excelente. Existen dos versiones, una de pago y una gratuita. La versión de pago tiene más funcionalidades como la posibilidad de poder ejecutar Jupyter Notebooks, los cuales son muy utilizados para aplicaciones en *cloud* e IA, y crear diagramas de clases UML. Además de las características ya comentadas, PyCharm cuenta con un *debugger* que tiene la posibilidad de introducir *breakpoints* o puntos de interrupción para depurar el código de manera muy sencilla y rápida. Por último, también cuenta con una integración de control de versiones Git y con las funcionalidades de creación, modificación y activación de entornos virtuales como Conda o virtualenv, todo ello sin salir del IDE [15].

Se ha hecho uso de un control de versiones: Git. Git guarda las modificaciones mediante confirmaciones (*commits*) en local, permitiendo una subida posterior (*push*) a un servidor Git que pueda gestionar la información. En este caso se ha utilizado GitHub, la cual es una solución de servidores Git muy popular que permite tener servidores públicos y privados de manera gratuita, aunque tiene una versión de pago que da más funcionalidades. Este servicio, Git, permite mantener un histórico de todas las versiones, con el añadido de poder volver atrás a una versión anterior con gran facilidad. Por tanto,

si algo falla tras un experimento realizado, se puede restaurar el estado previo sin perder trabajo. Git tiene otras funcionalidades añadidas, que en este caso no se han explotado, pero que en muchos casos profesionales son la razón por la que se escogen estos sistemas de control de versiones: trabajo colaborativo entre varios desarrolladores en el mismo repositorio y la creación de ramas aisladas de trabajo para probar nuevas funcionalidades que en cualquier momento se pueden fusionar con la rama principal, tras una revisión y resolución de las diferencias en los ficheros [16].

En último lugar, se hablará de los entornos virtuales. Son espacios aislados donde se pueden instalar los paquetes y las dependencias de un proyecto sin que entren en conflicto con el resto de los paquetes utilizados por otros proyectos y por el sistema global del dispositivo. Cada entorno virtual tiene su propio directorio donde guarda las librerías utilizadas y, en el caso de este proyecto, una versión específica de Python. Todo esto garantiza un menor número de problemas asociados a las dependencias entre los paquetes del sistema y facilita la reproducibilidad en otros sistemas. Hay diferentes herramientas comunes en el mercado para usar, como venv (incluido nativamente por Python) [17], virtualenv (solución de terceros para distintas versiones de Python) y Conda (gestor de paquetes con soporte multiplataforma) [18]. Se ha optado por el uso de Conda, ya que proporciona sus propios canales para la instalación de paquetes ya compilados y pre-empaquetados (conda-forge), los cuales incluyen un gran número de librerías de alto rendimiento (TensorFlow, PyTorch...) y herramientas muy utilizadas en ciencia de datos [19]. Cabe destacar, además, que Conda instala no solo paquetes de Python, por ejemplo, sino que también instala sus bibliotecas subyacentes, escritas en C y C++, por lo que se simplifica la instalación de paquetes con extensiones nativas, minimizando en número de errores provocados por las dependencias entre paquetes [18].

3 PROPUESTA TÉCNICA Y DESARROLLO

3.1 Estructura del flujo de la librería

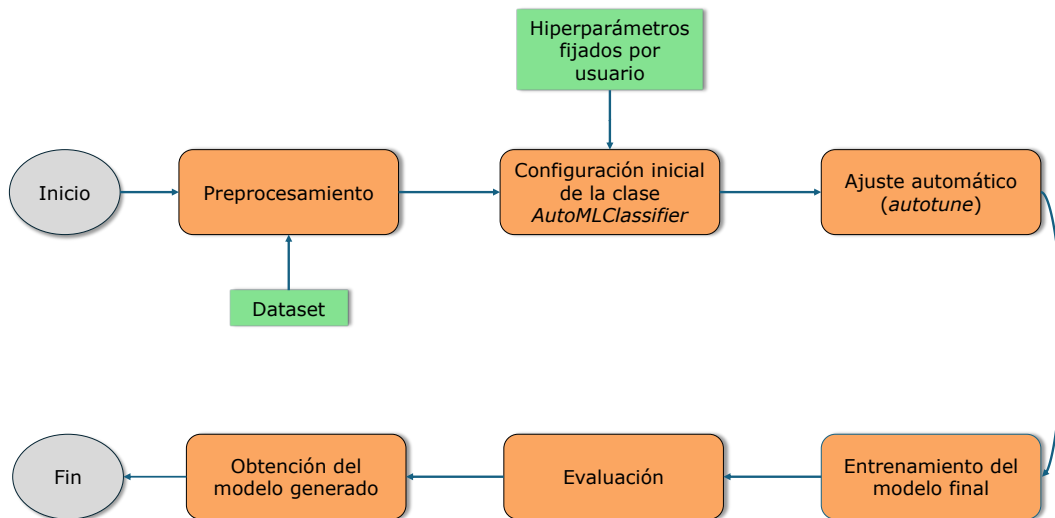


Figura 6. Estructura del flujo de uso de la librería. El color naranja representa procesos, mientras que el color verde representa entradas de datos a un proceso.

La Figura 6 muestra un diagrama de flujo con los principales pasos de ejecución de la librería desarrollada. La ejecución comienza con el preprocesamiento del *dataset* obtenido a partir del usuario, paso en el que el conjunto de datos se limpia y se prepara para funcionar correctamente con los algoritmos de Machine Learning utilizados. Una vez ya se tiene el *dataset* procesado, se configura la clase *AutoMLClassifier*. Esta clase representa el flujo automatizado que permite diseñar, afinar, entrenar y evaluar una red neuronal, permitiendo encontrar y establecer los mejores hiperparámetros para la misma. Opcionalmente, el usuario tiene la posibilidad de introducir valores fijados para algunos hiperparámetros que se usarán posteriormente en la búsqueda. Por ejemplo, el usuario puede decidir qué tasa de aprendizaje utilizar, por ejemplo, $1e^{-4}$, o incluso que la búsqueda del número de capas ocultas sea entre 2 y 6 capas. Esta configuración inicial prepara la clase para la búsqueda, inicializando valores por defecto si el usuario no los ha establecido, además de llevar a cabo la declaración de las variables internas que usará la librería.

Una vez instanciada la clase *AutoMLClassifier*, se procede a iniciar el ajuste automático, donde se empiezan a probar diferentes modelos y se acaba decidiendo cual de todos es el mejor, para un número determinado de pruebas. Cuando se han hallado los valores de los mejores hiperparámetros, se entrena el modelo resultante con todos los datos del conjunto de entrenamiento. Además, el usuario tiene la posibilidad de evaluar el modelo contra los datos de test y obtener la pérdida y la precisión del modelo antes de finalmente recuperar el modelo entrenado.

3.2 Preprocesamiento

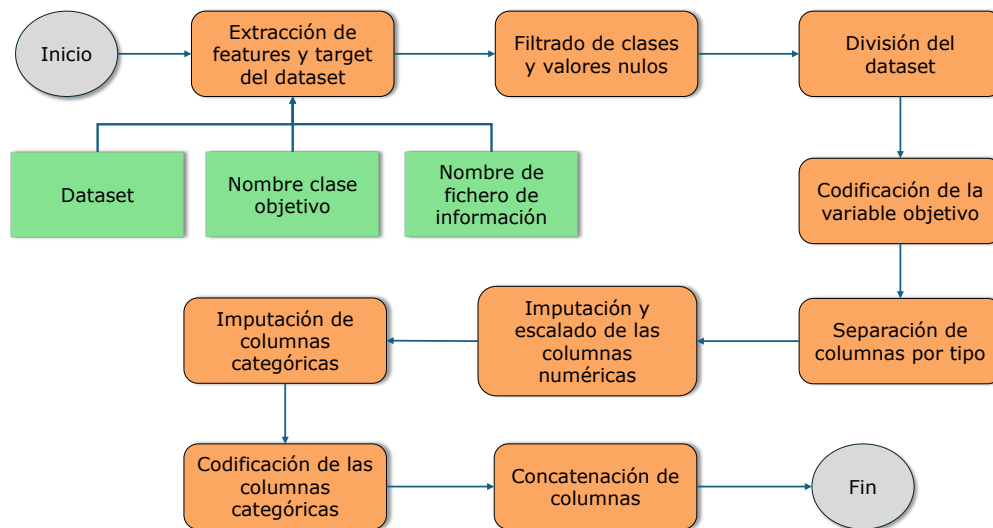


Figura 7. Diagrama de flujo del proceso de preprocesamiento de un dataset.

Los distintos pasos que se llevan a cabo durante el preprocesamiento de los datos pueden verse en la Figura 7. Este bloque de preprocesamiento ofrece al usuario una solución para que de manera automática se limpie su conjunto de datos y se prepare para que los algoritmos puedan trabajar con esos datos de manera correcta. Además, cabe mencionar que se crea un fichero que guarda información acerca de las características del *dataset*, cuyo objetivo es registrar información clave del preprocesamiento para fines de trazabilidad, análisis y comprensión del conjunto de datos. En el fichero se guarda la siguiente información: distribución de clases en el *dataset* (en porcentaje), número de clases objetivo y sus posibles valores, número de filas y columnas del *dataset*, existencia

o no de valores nulos en el conjunto de datos final ya procesado, así como el orden y nombre de las columnas originales del *dataset*. Se puede introducir la ruta a este fichero de manera opcional, y, en caso contrario, se escoge la ruta “./info_datasets”.

En cuanto al flujo de preprocesamiento, en primer lugar, se hace una extracción de las *features* y el *target* del *dataset* que se recibe como entrada por parte del usuario. Es decir, se distinguen o separan los datos de las etiquetas. Se espera que el usuario indique obligatoriamente el nombre de la columna objetivo, ya que dependiendo del *dataset*, el nombre de esta columna puede variar. Además, el *dataset* de entrada debe contener toda la información, es decir, tanto los datos como las etiquetas para que posteriormente el código utilice toda la información para dividirla. Seguidamente, el siguiente paso es el filtrado de clases y valores nulos, donde se eliminan las filas que pertenecen a una clase con menos de dos filas de representación en el *dataset*, ya que se necesita tener al menos dos instancias de la misma clase objetivo para poder hacer un balance de clases en el conjunto de datos de entrenamiento y de test en el proceso de división del *dataset* que se llevará a cabo posteriormente. Esta técnica, que se conoce como estratificación, permite balancear las clases objetivo del *dataset* consiguiendo que en ambos subconjuntos existan instancias o filas para cada clase objetivo, lo cual incrementa la fiabilidad de los resultados de la validación de una red neuronal. Además, se eliminan las filas que contengan un *target* nulo, ya que no se pueden utilizar de manera fiable para entrenar un modelo.

Después, se hace la división del conjunto de datos en entrenamiento y test, para lo cual se usa una función de scikit-learn denominada “train_test_split()”. Esta función permite indicar el porcentaje de datos del *dataset* que serán utilizados para construir el conjunto de test, que en este caso es un 20% para el test y el 80% restante para entrenamiento. Retorna un conjunto de entrenamiento con los datos sin etiquetar: X_{train} , otro conjunto con las etiquetas de los datos de X_{train} : y_{train} , y lo mismo para los datos y etiquetas de test: X_{test} e y_{test} . Cabe resaltar que esta función acepta un parámetro para indicar el uso de “stratify”, indicando que quiere emplearse la técnica de estratificación.

El resto de los procesos que se explican en esta sección llevan a cabo una transformación de las columnas de los conjuntos de datos para que tengan una representación adecuada

para algoritmos de aprendizaje automático. El primer paso consiste en codificar las columnas que contienen las etiquetas, ya que normalmente los modelos de redes neuronales que resuelven problemas de clasificación multiclase (con varias clases objetivo) lo necesitan. Para ello, se ha usado sobre el conjunto y_{train} e y_{test} una técnica de codificación denominada *One-Hot Encoding*, la cual convierte cada clase en un vector binario donde solo una posición es 1 y el resto son 0 (ver Figura 8).

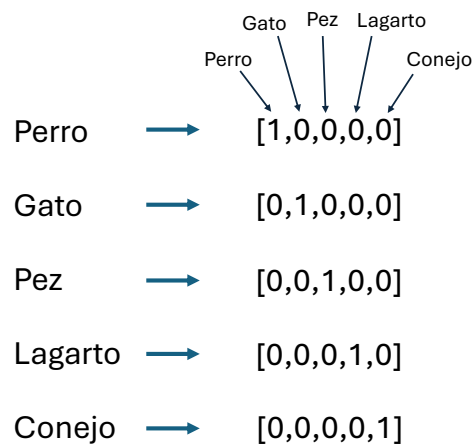


Figura 8. Ejemplo de codificación One-Hot Encoding para clasificación multiclase.

Con esta codificación realizada, las etiquetas ya están listas para los algoritmos posteriores. En cuanto a los subconjuntos de datos sin etiquetar (entrenamiento y test), el siguiente paso es hacer una separación entre las columnas que contienen datos numéricos y las que contienen datos categóricos, ya que se procesarán de manera distinta. Las columnas numéricas se imputan con la media, para rellenar los valores de los campos que no estén completos. Después, las columnas se escalan, ya que las redes neuronales son sensibles a la magnitud de las variables. Por ejemplo, si existe una columna que tiene un valor mínimo y máximo de 0 y 3, y otra columna de 0 y 30.000, la segunda columna, como tiene un rango de valores mucho más extenso, puede ser asignada una mayor importancia a vistas de los pesos de la red neuronal, solamente debido a sus escalas, provocando que la red no se comporte de manera óptima. Por ello, se utilizan técnicas de estandarización, como el escalado, que permite evitar que columnas con rangos grandes

dominen el proceso de optimización. Para este trabajo, se ha utilizado *StandardScaler*, que permite realizar una transformación sobre los datos para que tengan media 0 y desviación típica 1. En la práctica, cada variable x se convierte en $x' = \frac{x-\mu}{\sigma}$, donde μ y σ son la media y la desviación típica. Gracias a esta transformación, las variables de las columnas quedan centradas en 0 y normalizadas a la misma escala.

Las columnas categóricas, en cambio, se imputarán utilizando la moda de la columna, es decir, el valor más frecuente de esa columna, y seguidamente se codificarán utilizando de nuevo *One-Hot Encoding*, ya que al igual que se ha comentado previamente, estas columnas tienen que ser convertidas a valores numéricos que la red neuronal pueda interpretar. Una vez ya están todas las columnas procesadas adecuadamente, se procede a concatenar las columnas numéricas y categóricas de los *datasets* de datos sin etiquetar para volver a unificar los datos (recordemos que se habían separado las columnas categóricas y numéricas para su procesamiento independiente) y finalmente se devuelve al usuario todos los subconjuntos de datos creados: X_{train} , X_{test} , y_{train} e y_{test} . De esta manera, el usuario puede conseguir el preprocesamiento de su conjunto de datos de forma automática sin necesidad de ningún conocimiento sobre ello.

3.3 Inicialización clase *AutoMLClassifier*

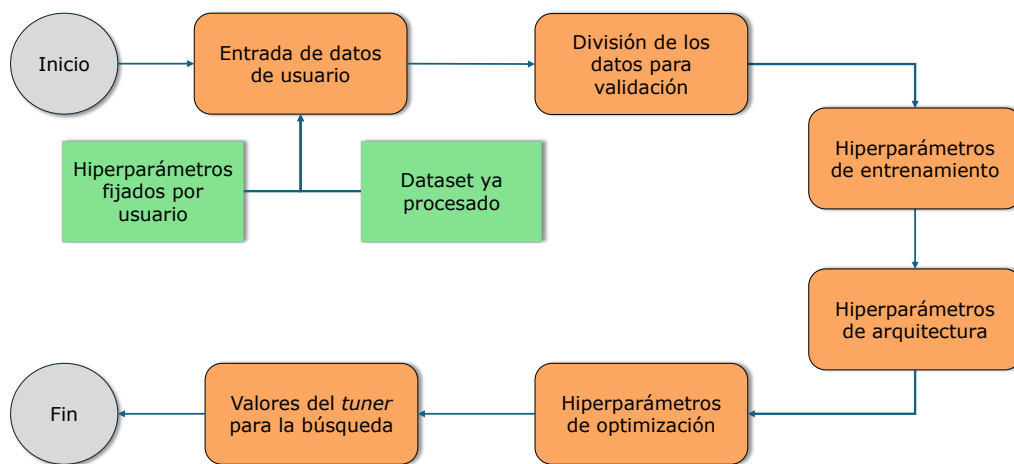


Figura 9. Diagrama de flujo del proceso de inicialización de la clase *AutoMLClassifier*.

La clase *AutoMLClassifier* es la encargada de llevar a cabo la búsqueda de los mejores hiperparámetros de una red neuronal para el *dataset* que proporciona el usuario. En primera instancia, la clase se inicializa, lo cual permite instanciar el entorno de búsqueda, incluyendo rangos de hiperparámetros por defecto. La inicialización sigue los pasos mostrados en la Figura 9.

Se comienza con la recolección de los datos que el usuario pasa por parámetro en la creación de un objeto de la clase *AutoMLClassifier*. Estos datos son necesarios para poder personalizar el entrenamiento del modelo en caso de que el usuario quiera introducir valores fijos. Seguidamente, se establece el valor del porcentaje de los datos que se usarán para validación y entrenamiento. La validación consiste en separar en otro subconjunto un porcentaje de los datos que se usarían para entrenar el modelo, permitiendo observar tras cada época de entrenamiento cómo se comporta el modelo ante datos que no ha visto durante su entrenamiento. Esta técnica permite tunear los hiperparámetros sin modificar el conjunto de datos de test, que solamente deben ser utilizados para verificar el comportamiento final del modelo una vez entrenado. Para este caso de uso, se ha seleccionado que un 30% de los datos de entrenamiento serán dedicados a la validación. Cabe destacar que el usuario tiene la posibilidad de pasar a la clase su propio conjunto de datos de validación. En caso contrario, se dividirá el conjunto de datos de entrenamiento consiguiendo ambos subconjuntos (aplica tanto a X_{train} , como a y_{train}). Es decir, al final de este proceso se contará con cuatro conjuntos de datos: X_{val} , X_{train} , y_{val} e y_{train} (ver Tabla 1).

Hiperparámetro	Valor por defecto	Función / Comentario
División de validación	0.30	30 % de los datos de entrenamiento se reservan para validación
X_{train} , X_{val} , y_{train} , y_{val}	70% entrenamiento / 30% validación	Si no se pasa X_{val}/y_{val} , se generan con <code>train_test_split</code>

Tabla 1. Resumen de valores de hiperparámetros asociados a la división del conjunto de datos en entrenamiento, test y validación.

Los siguientes hiperparámetros que se inicializarán son los relacionados al entrenamiento del modelo. Se establece el tamaño del *batch*, en el cual, si el usuario no introduce un valor fijo, se optará por un tamaño de 16 muestras. Después, se inicializa para su uso el “*EarlyStopping*”, técnica de regularización que consiste en parar el

entrenamiento final del modelo escogido si se detecta que, entre época y época de entrenamiento, no existe una mejora de la métrica monitorizada (pérdida, precisión, etc...) de manera seguida durante ciertas épocas. Con esta técnica se consigue evitar el sobreajuste del modelo y detener el entrenamiento cuando se detecta que el modelo ya no mejora (dando lugar a un ahorro de tiempo y recursos). Esta técnica consta de varios parámetros que modifican su comportamiento:

- *Min_delta*: Es la variación mínima en la métrica monitorizada que se considera una mejora. En otras palabras, si entre dos épocas de entrenamiento, el valor de la métrica monitorizada varía como mínimo el valor de *min_delta*, se considera como que ha habido una mejora en la métrica. En caso contrario, es decir, si la variación de la métrica entre dos épocas ha sido menor que *min_delta*, se dice que en esas épocas no existido mejora de la métrica. Con respecto a la librería desarrollada en este trabajo, la métrica monitorizada para evaluar el *EarlyStopping* es la pérdida sobre el conjunto de datos de validación, por lo que, se considerará una mejora si esta disminuye una magnitud mínima de *min_delta*. Se ha determinado que un buen valor de variación mínima para los modelos estudiados es de $1e^{-4}$.
- *Paciencia (o patience en inglés)*: Es el número de épocas consecutivas sin mejora que tienen que transcurrir para que se pare el entrenamiento, debido a que no se observa una mejora en la métrica objetivo. Se ha utilizado un valor de 10 épocas consecutivas sin mejora antes de parar el entrenamiento.
- *Opción restore_best_weights*: Esta opción permite guardar los pesos del modelo de la época en que mejor métrica de validación (en este caso la pérdida sobre el conjunto de datos de validación) se obtuvo durante todo el entrenamiento hasta el momento de la parada.

Después, se inicializa el valor del número de épocas que entrenará el modelo final escogido, es decir, una vez se hayan buscado los mejores valores para los hiperparámetros. Se han escogido 1000 épocas, un valor bastante alto para conjuntos de datos sencillos, pero, dado que el modelo final se entrenará usando *EarlyStopping*, este detendrá el entrenamiento si ya no hay mejora en cualquier momento antes de llegar a las 1000 épocas.

Por último, se establece la métrica de pérdida que se calculará durante el entrenamiento, utilizándose *binary_crossentropy* si el número de clases objetivo es dos, o *categorical_crossentropy* si el número de clases objetivo es mayor de dos. Además, es importante destacar que el uso de *categorical_crossentropy* es debido a que las etiquetas o clases objetivo están codificadas utilizando *One-Hot Encoding*, ya que esta métrica de pérdida espera este tipo de codificación de valores. En el caso de no tener codificación de las clases y simplemente ser, por ejemplo, números enteros, se usaría la versión *sparse*, que trabaja sobre clases no codificadas (ver Tabla 2).

Hiperparámetro	Valor por defecto	Función / Comentario
Tamaño del batch	16	Número de muestras por batch
Número de épocas de entrenamiento del modelo final	1000	Máximo de épocas (detenido antes por EarlyStopping)
EarlyStopping	patience=10, min_delta=1e-4, restore_best_weights=True	Detiene si no mejora la métrica de validación
Métrica de pérdida	"binary_crossentropy" si 2 clases, "categorical_crossentropy" si >2	Depende de One-Hot vs. clases enteras

Tabla 2. Resumen de valores de hiperparámetros asociados al entrenamiento del modelo.

En cuanto a los hiperparámetros de arquitectura del modelo, primero se inicializa la tasa de aprendizaje. Si el usuario no ha introducido un valor fijo a usar, se le asigna un valor de $1e^{-5}$, ya que debe tener un valor por defecto en caso de que se necesite para construir un modelo para alguna búsqueda. Es decir, hasta que no se realiza la búsqueda de la tasa de aprendizaje, esta tendrá el valor de $1e^{-5}$ o el valor introducido por el usuario. El proceso de búsqueda o “*autotune*” se explica en profundidad en la siguiente subsección (ver 3.4). A su vez, se inicializan los valores mínimos y máximos para la búsqueda de la tasa de aprendizaje, estableciendo como valor mínimo: $1e^{-6}$ y valor máximo: $1e^{-2}$.

A continuación, se inicializan los valores asociados a la búsqueda de la función de activación de las capas ocultas. Como el algoritmo utilizará una lista de funciones de activación e irá iterando sobre ella buscando la función que mejor resultado dé, el usuario tiene la opción de pasar en el proceso de entrada de datos una lista con los nombres de las funciones de activación que desea probar. En el caso de que no introduzca valor, se probarán por defecto las funciones: *relu*, *elu* y *silu*. Cabe destacar que el nombre de las funciones que se introduzca por parte del usuario debe coincidir con el nombre usado por Keras en su librería.

En cuanto al número de neuronas de la capa de entrada y de salida, estas seguirán la estructura definida por el *dataset*, donde la capa de entrada tendrá una neurona por cada variable de entrada y la capa de salida tendrá una neurona por cada clase a predecir. Después, se escoge la función de activación de la capa de salida de la red neuronal. Al igual que ocurre con la métrica de pérdida, si el número de clases objetivo es dos, se usará *sigmoid*, mientras que, si es mayor que dos se usará *softmax*. La función *sigmoid* (sigmoide en español) está especializada en problemas de clasificación binaria, dado que su resultado es un escalar entre (0,1), mientras que *softmax* funciona bien para clasificación multiclase con *One-Hot Encoding* porque su salida es un vector binario.

Seguidamente, se inicializan los hiperparámetros relacionados con la búsqueda del número de neuronas por cada capa oculta del modelo. Se establece un valor inicial de 10 neuronas por capa oculta (es decir, para poder crear un modelo válido para la búsqueda del número de capas ocultas, ya que es la primera búsqueda realizada. Si no, el modelo daría error al no tener un valor definido de número de neuronas por capa oculta). Se definen 5 neuronas como límite inferior y $\left\lceil \frac{2}{3}d + c \right\rceil$ neuronas como límite superior, donde d es el número de características o columnas del *dataset* ya procesado y c es el número de clases objetivo. Además, se establece un umbral de 100 neuronas por capa, a partir del cual la búsqueda del hiperparámetro se haría de manera logarítmica, con el objetivo de probar un mayor rango de valores posibles. Por otra parte, se da la posibilidad al usuario de introducir el valor mínimo y máximo entre los que realizar la búsqueda. El código analiza los valores introducidos en busca de errores lógicos en ellos (ejemplo: El valor máximo debe ser mayor que el valor mínimo) y establece el rango de búsqueda correcto en función de ello.

En cuanto al número de capas ocultas, el valor inicial establecido es nulo, ya que será el primer hiperparámetro a buscar, lo cual significa que el *tuner* le asignará un valor del rango y por tanto siempre tendrá un valor y no dará error. Si el usuario no especifica un rango determinado, el número de capas ocultas se explorará entre un valor mínimo de 2 y un máximo de 6. Al igual que con el número de neuronas por capa oculta, el código analiza los rangos introducidos por el usuario y determina si son correctos (ver Tabla 3).

Hiperparámetro	Valor por defecto	Función / Comentario
Tasa de aprendizaje	1,0E-05	Tasa de aprendizaje inicial antes de autotune
Mínima tasa de aprendizaje	1,0E-06	Límite inferior para el rango de búsqueda de la tasa de aprendizaje
Máxima tasa de aprendizaje	1,0E-02	Límite superior para el rango de búsqueda de la tasa de aprendizaje
Número de neuronas de capa de entrada	(n_features,)	Una neurona por cada variable de entrada
Número de neuronas de capa de salida	n_classes	Una neurona por cada clase a predecir
Función de activación de capas ocultas	"relu", "elu" y "silu"	Funciones de activación por defecto
Función de activación de capa de salida	"softmax" si >2 clases, "sigmoid" si 2 clases	Mapeo según tipo de clasificación
Número de neuronas por capa oculta	10	Valor por defecto necesario para realizar la primera búsqueda
Mínimo número de neuronas por capa oculta	5	Límite inferior para el rango de búsqueda del número de neuronas por capa oculta
Máximo número de neuronas por capa oculta	$\lceil 2/3 \cdot d + c \rceil$	Límite superior para el rango de búsqueda del número de neuronas por capa oculta
Umbral de búsqueda logarítmica para neuronas por capa oculta	100	Número de neuronas a partir del cual se hace búsqueda logarítmica
Número de capas ocultas	None	Es la primera búsqueda. No hace falta que se inicialice valor alguno
Mínimo número de capas ocultas	2	Límite inferior para el rango de búsqueda del número de capas ocultas
Máximo número de capas ocultas	6	Límite superior para el rango de búsqueda del número de capas ocultas

Tabla 3. Resumen de valores de hiperparámetros asociados a la arquitectura del modelo.

d es el número de características y *c* es el número de clases objetivo.

Acto seguido, se inicializan los valores asociados a la búsqueda del optimizador. Como el código espera una lista de funciones que representan optimizadores de Keras, el usuario puede introducir esta lista por parámetro en la entrada de datos. En su defecto, se prueba con los siguientes optimizadores: *adam*, *sgd* y *rmsprop*. Además, como el código realiza una búsqueda de los mejores valores de los hiperparámetros internos de cada optimizador, estos se inicializan a nulo (ejemplo de hiperparámetros internos: *beta1*, *beta2*, *nesterov*, etc.). Adicionalmente, el código contiene un diccionario de mapeo o traducción entre el nombre común de los optimizadores (el nombre que Keras utiliza internamente en sus librerías) y la función que representan, por lo que el usuario solo debe introducir, por ejemplo, *adam* y el código lo traducirá automáticamente a “tf.keras.optimizers.Adam” para su uso como optimizador de Keras (ver Tabla 4).

Hiperparámetro	Valor por defecto	Función / Comentario	Optimizador asociado
Lista de optimizadores	"adam", "sgd", "rmsprop"	Optimizadores por defecto	N/A
Mínimo valor de beta1	0.85	Límite inferior para el rango de búsqueda del valor de beta1	adamw, adamax, lion, adam, nadam
Máximo valor de beta1	0.99	Límite superior para el rango de búsqueda del valor de beta1	adamw, adamax, lion, adam, nadam
Mínimo valor de beta2	0.85	Límite inferior para el rango de búsqueda del valor de beta2	adamw, adamax, lion, adam, nadam
Máximo valor de beta2	0.9999	Límite superior para el rango de búsqueda del valor de beta2	adamw, adamax, lion, adam, nadam
Clipnorm	1.0	Valor por defecto de clipping de gradientes	Todos los optimizadores
Mínimo valor de rho	0.8	Límite inferior para el rango de búsqueda de rho	adadelata, rmsprop
Máximo valor de rho	0.95	Límite superior para el rango de búsqueda de rho	adadelata, rmsprop
Mínimo valor de momentum	0.7	Límite inferior para el rango de búsqueda de momentum	rmsprop, sgd
Máximo valor de momentum	0.9 si optimizador == "rmsprop" , 0.95 si optimizador == "sgd"	Límite superior para el rango de búsqueda de momentum	rmsprop, sgd
Nesterov	True	Valor por defecto de Nesterov	sgd

Tabla 4. Resumen de valores de hiperparámetros asociados al optimizador.

En cuanto a la inicialización de los valores internos del *tuner* para las búsquedas, se empieza con la elección del número de épocas que entrenará cada *trial*. Para consultar cómo funciona el *tuner* en profundidad se debe consultar la siguiente subsección 3.4. Si el usuario no introduce ningún valor específico, se usarán 5 épocas de entrenamiento por *trial*. En cuanto al número de *trials* por cada etapa de búsqueda, si no hay un valor especificado por el usuario, se escogerá usar 5 *trials*. Pero, para la etapa de búsqueda de la función de activación, el número de *trials* es diferente, será igual a la longitud de la lista de funciones de activación a probar. De esta manera, se hace una prueba con cada función y se valora cual es la que mejor funciona. No tiene sentido probar más de una vez con la misma función de activación, ya que el modelo no cambia. Con la búsqueda del optimizador ocurre algo parecido, ya que, como mínimo, el número de *trials* será la longitud de la lista de optimizadores, con la posibilidad de que, si el usuario ha establecido

un valor de *trials* mayor que la longitud de la lista, el que se usará será el definido por el usuario, debido a que, la búsqueda se realiza a la vez sobre el optimizador y la tasa de aprendizaje. En este caso, sí tiene sentido permitir un mayor número de *trials* por encima de la longitud de la lista, porque el valor de la tasa de aprendizaje va cambiando, lo que produce que, aunque se use el mismo optimizador en dos *trials* distintos, el modelo cambiará al tener una tasa de aprendizaje diferente, por tanto, sí que pueden generar resultados diferentes.

Finalmente, se establece el objetivo del *tuner*, o lo que es lo mismo, la métrica por la que decidirá cuál es el mejor valor de entre los resultados obtenidos para una etapa de búsqueda. Esta métrica será la pérdida sobre el conjunto de datos de validación, ya que es más fiable para comparar el comportamiento que la pérdida sobre el conjunto de entrenamiento, debido a que los datos de validación son datos sobre los que el modelo no ha sido entrenado. Los resultados de ejecución del *tuner* se guarden bajo el directorio “autotune” (ver Tabla 5).

Hiperparámetro	Valor por defecto	Función / Comentario
Número de épocas de entrenamiento por trial	10	Valor por defecto de épocas a entrenar en un trial
Número de trials	15	Número de modelos a probar por búsqueda
Número de trials para búsqueda de la función de activación	Igual a la longitud de la lista de funciones de activación a probar	Número de modelos a probar para la búsqueda de la función de activación
Número de trials para búsqueda de optimizador	Como mínimo igual a la longitud de la lista de optimizadores	Número de modelos a probar para la búsqueda de optimizador
Objetivo	Pérdida sobre el conjunto de validación (val_loss)	Métrica de decisión interna del tuner

Tabla 5. Resumen de valores de hiperparámetros asociados al tuner.

3.4 Ajuste automático (autotune)

El ajuste automático (o *autotune*, en inglés) es el proceso por el cual se van a realizar las búsquedas de los mejores hiperparámetros. A grandes rasgos, el proceso consiste en crear un modelo de red neuronal con unos hiperparámetros determinados que irán cambiando y se irá probando cuáles funcionan mejor. Si el usuario ha fijado un valor

concreto para algún hiperparámetro, no será necesario hacer ninguna búsqueda para ese hiperparámetro y se pasará directamente a la etapa de búsqueda del siguiente. Es decir, para cada etapa de búsqueda, se creará un modelo de red neuronal que dejará fijos todos los hiperparámetros, excepto el que se está buscando, el cual con cada *trial* irá cambiando de valor para evaluar cual es el mejor valor de entre todos los probados. El orden de búsqueda es el siguiente:

1. Número de capas ocultas y tasa de aprendizaje
2. Número de neuronas por capa oculta
3. Función de activación de las capas ocultas
4. Optimizador y tasa de aprendizaje
5. Reajuste de la tasa de aprendizaje
6. Hiperparámetros internos del optimizador

Es importante destacar que, en las búsquedas, es necesario tener un modelo de red neuronal completo para que el entrenamiento funcione y para que no salte un error de creación del modelo, por lo que es necesario asumir valores de otros hiperparámetros por defecto antes de que hayan sido seleccionados. Es decir, si, por ejemplo, el código se encuentra en la primera etapa (número de capas ocultas y tasa de aprendizaje), el modelo necesita tener asignado un optimizador, ya que saltará error si el modelo no tiene un optimizador para entrenar, por lo que se asume un optimizador por defecto. Más tarde, cuando llegue la etapa de búsqueda del optimizador, este cambiará y se evaluarán los posibles valores que puede tener y cual es realmente el mejor valor para el *dataset* concreto. Esto se debe a que se ha optado por hacer las etapas de búsqueda de manera secuencial, en vez de hacer una sola etapa. Si se hubiesen hecho todas las búsquedas en una sola etapa, el *tuner* podría modificar todos los hiperparámetros a la vez (es decir, hacer una sola búsqueda que incluya todos los hiperparámetros). En ese caso, la complejidad computacional del programa se incrementaría, ya que los posibles valores a buscar pasan a ser todas las combinaciones posibles de los hiperparámetros a la vez. Es decir, si se tienen n hiperparámetros y cada uno admite k_n posibles valores, el primer enfoque (división de búsquedas por etapas) da lugar a un coste total de barrido exhaustivo de $\sum_{i=1}^n k_i$, mientras que el segundo enfoque (una única búsqueda) tendría un coste de $\prod_{i=1}^n k_i$. Por tanto, se observa como el crecimiento de la complejidad es exponencial y

para valores moderados de k y n el problema se volvería inabordable. La estrategia secuencial de optimización de hiperparámetros sigue heurísticas basadas en experiencia y en la literatura científica, y es una de las principales contribuciones de este trabajo. El algoritmo desarrollado que describe la resolución del código de las búsquedas usando el enfoque de división de las búsquedas por etapas se resume de la Figura 10.

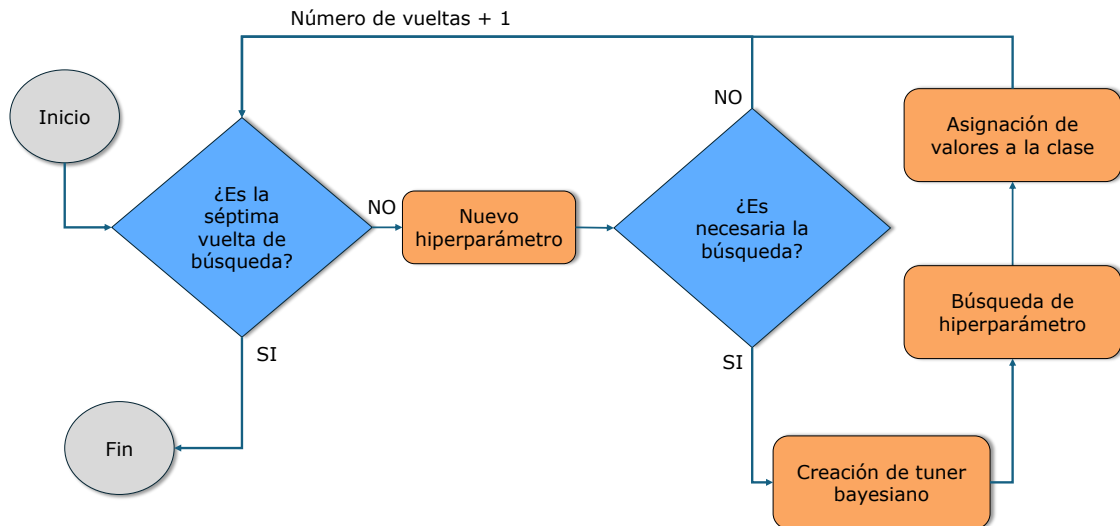


Figura 10. Diagrama de flujo del proceso de ajuste automático de los hiperparámetros de un modelo de red neuronal. El color azul en el diagrama representa una toma de decisiones.

El algoritmo se puede observar como un bucle, en el cual, por cada iteración, se analiza si hace falta realizar una búsqueda para el hiperparámetro en específico (siguiendo el orden de búsqueda detallado con anterioridad) y si es así, se crea el *tuner* que realizará la búsqueda y se lleva a cabo. Después, se asigna el mejor valor encontrado como atributo de la clase “*AutoMLClassifier*”, que guarda toda la información acerca del algoritmo, y se repite el proceso con las 6 etapas de búsqueda posibles de realizar.

A continuación, se explicará en profundidad la creación y el funcionamiento del *tuner*. Es necesario entender cómo funciona la librería Keras Tuner internamente. Utiliza tres componentes principales: *HyperModel*, *Oracle* y *Tuner*. Según la documentación oficial de la librería, el *HyperModel* es una clase que tiene un método (*build*) que recibe un objeto con el hiperespacio de valores de hiperparámetros posibles y que a su vez crea

un modelo de Keras utilizando el propio hiperespacio para asignar los valores correspondientes. Esta clase solamente configura el modelo según las elecciones de valores del hiperespacio de la etapa concreta [20].

El *Oracle* contiene la inteligencia, dicho de otra manera, es el que decide que valores de hiperparámetros probar, dentro del rango establecido. Existen varios oráculos disponibles en Keras Tuner: *RandomSearch*, *BayesianOptimization* y *Hyperband*. El primero selecciona valores o combinaciones de valores al azar, mientras que el segundo modela la función objetivo (en nuestro caso, la pérdida sobre los datos de validación) como una gaussiana para predecir posibles valores buenos que probar. La última opción, *Hyperband*, ejecuta varias posibles configuraciones y detiene las que funcionan mal pronto para dedicar más recursos a las que funcionan mejor [21]. Por último, el *Tuner* es la pieza que combina los otros dos elementos, ya que cuando se realiza una búsqueda, el *Tuner* es el encargado de que el *Oracle* pida un *trial*, es decir, los valores para una iteración sobre el modelo. Después, el *Tuner* llama al método *build* que tiene el *HyperModel*, creando el modelo con los nuevos valores que ha pedido el *Oracle*. Por último, el *Tuner* es el encargado de entrenar el modelo, recoger la métrica objetivo y repetir este proceso para el número de *trials* establecido [22]. Además, cabe resaltar que el *Tuner* incorpora un método que permite devolver los valores de los mejores resultados en función del objetivo establecido: *get_best_hyperparameters*. Con ello, se realiza la asignación de los valores a los atributos de la clase *AutoMLClassifier*. Con respecto al *Oracle* escogido, se ha optado por utilizar un oráculo de tipo *BayesianOptimization*; habitualmente esta opción tiene los mejores resultados al ir acumulando evidencia de los *trials* anteriores para informar la siguiente elección de hiperparámetros.

3.5 Entrenamiento del modelo final, evaluación y selección de mejores hiperparámetros

Una vez se ha realizado el *autotune* y se han guardado los resultados en la clase *AutoMLClassifier*, el siguiente paso consiste en entrenar el modelo resultante con todos los datos de entrenamiento. Este modelo contiene los mejores valores encontrados para los diferentes hiperparámetros que se han buscado. Para entrenarlo, se llama al método *fit* que ofrece la librería Keras (tf.keras), a la cual se le pasa como parámetros para el

entrenamiento los conjuntos de entrenamiento (X_{train}, y_{train}), los conjuntos de validación, el número de épocas a entrenar (recordemos que se estableció en 1000 épocas, debido al uso de *EarlyStopping*), el tamaño de *batch* a utilizar y por último se indica que se use *EarlyStopping*. Una vez finalizado el entrenamiento, el modelo ya contiene los pesos de las neuronas actualizados y estaría listo para ser utilizado sobre nuevas instancias.

Acto seguido, se procede a evaluar el modelo entrenado, para lo cual se invoca al método *evaluate* que ofrece Keras, el cual recibe como parámetros los conjuntos de test y el tamaño del *batch*. Este método devuelve las métricas de precisión y pérdida sobre el conjunto de datos de test, verificando que el modelo funcione igual de bien o no para instancias con las que no ha sido entrenado previamente. Es una manera, por tanto, de evaluar la capacidad de generalización del modelo, así como de detectar si existe sobreajuste, es decir, que el modelo haya aprendido patrones que se repiten en el conjunto de datos de entrenamiento, pero que no existen fuera de ese conjunto, y, como consecuencia, falle al intentar predecir valores del conjunto de test. Finalmente, después de haber ejecutado el entrenamiento final del modelo, se le ofrece al usuario un método que devuelve ese modelo para su uso.

3.6 Validación

La comprobación del funcionamiento de la librería se ha llevado a cabo a través de pruebas con distintos conjuntos de datos. Se han utilizado conjuntos de datos extraídos de las plataformas de *ucimlrepo* (*UC Irvine Machine Learning Repository*) y Kaggle, las cuales son repositorios de *datasets* comúnmente conocidos para su uso en la disciplina de aprendizaje automático. En la Tabla 6 pueden verse el número inicial de instancias, el número de instancias tras el preprocesamiento, el número de columnas y el tipo de dato de dichas columnas para los conjuntos de datos empleados en la validación de la librería.

Además, para la correcta validación de la librería, se necesitaría información acerca de la precisión máxima a la que un modelo de red neuronal óptimo podría llegar para estos datasets. Esta información está disponible en *ucimlrepo* para algunos datasets, por lo que se utilizará posteriormente para comparar los resultados obtenidos (ver Tabla 7).

Nombre	Número de instancias antes de procesar	Número de instancias después de procesar	Número de columnas	Tipo de columnas
Bank Marketing	45211	36128	16	Numéricas y categóricas
Breast Cancer Wisconsin	569	455	30	Solo numéricas
Car Evaluation	1728	1382	6	Numéricas y categóricas
Census Income	48842	26048	14	Numéricas y categóricas
Default of Credit Card Clients	30000	24000	24	Solo numéricas
Dry Bean	13611	10888	16	Solo numéricas
Heart Disease	303	242	12	Solo numéricas
MAGIC Gamma Telescope	19020	15216	11	Solo numéricas
Mushroom	8124	6499	22	Numéricas y categóricas
Spambase	4601	3680	57	Solo numéricas

Tabla 6. Resumen de características de los distintos datasets utilizados para la validación de la librería.

Nombre	Precisión máxima alcanzable por un modelo de red neuronal (ucimlrepo)
Bank Marketing	No existe información asociada
Breast Cancer Wisconsin	92,30%
Car Evaluation	98,30%
Census Income	No existe información asociada
Default of Credit Card Clients	No existe información asociada
Dry Bean	No existe información asociada
Heart Disease	78,90%
MAGIC Gamma Telescope	No existe información asociada
Mushroom	100%
Spambase	91,49%

Tabla 7. Información ofrecida por ucimlrepo sobre precisión máxima alcanzable por un modelo de red neuronal de clasificación para cada dataset utilizado [23-27].

4 RESULTADOS Y DISCUSIÓN

4.1 Ejemplo de uso

A continuación, se ilustrará cómo funciona la librería con un ejemplo de código. En primer lugar, antes de utilizar la librería, el usuario debe tener un *dataset* cargado para su uso. En este caso, se ha realizado una simulación de obtención de un *dataset* a través del repositorio de Kaggle, desde el cual se ha importado el *dataset* Mushroom (líneas 7-14). Además, a través de la información de Kaggle sobre el *dataset*, se determina que la clase objetivo, es decir, la columna que contiene las etiquetas se denomina *class*. Una vez se tiene el *dataset* cargado, se instancia la clase *AutoMLClassifier*, cuyo constructor recibe como parámetros solamente el *dataset* y el nombre del fichero donde se guardará información acerca del *dataset* una vez procesado. Después, simplemente se llama al método “autotune()” para realizar la búsqueda completa de hiperparámetros y al método “train()” para entrenar el modelo resultante de la búsqueda. Seguidamente, el usuario tiene la opción de llamar el método “evaluate()” para evaluar la precisión y la pérdida del modelo (no es obligatorio). Finalmente, se obtiene el modelo entrenado resultante a través de la llamada al método “get_final_model()”.

```

1  from pathlib import Path
2
3  import kagglehub
4  import pandas as pd
5  from AutoMLClassifier import AutoMLClassifier
6
7  ruta_fichero_kaggle = "uciml/mushroom-classification"
8  ruta_fichero_csv_interno = "mushrooms.csv"
9  nombre_fichero_dataset = "mushroom"
10 nombre_clase_objetivo = "class"
11
12 dataset_dir = kagglehub.dataset_download(ruta_fichero_kaggle)
13 dataset_path = Path(dataset_dir)
14 data = pd.read_csv(dataset_path / ruta_fichero_csv_interno)
15
16 model_search = AutoMLClassifier(data, clase_objetivo: "class", nombre_fichero_info_dataset: "mushroom")
17 model_search.autotune()
18 model_search.train()
19 loss,precision=model_search.evaluate()
20 final_model = model_search.get_final_model()

```

Figura 11. Ejemplo de uso de la clase *AutoMLClassifier* para autotuning, entrenamiento, evaluación y obtención del modelo resultante.

El resultado de la ejecución del código muestra por pantalla información acerca de la arquitectura del modelo, así como de la precisión y pérdida que se ha obtenido al evaluarlo (ver Figura 12). En este caso, el modelo ha obtenido una precisión de 100% utilizando los valores de búsqueda por defecto de la clase de *autotune*. Para este *dataset*, se ha obtenido, por tanto, un modelo con una precisión acorde a la que indica ucimlrepo (ver Tabla 7).

```
##### Resultados obtenidos tras el entrenamiento final #####
Número de capas ocultas: 6
Número de neuronas por capa oculta: 6
Función de activación: relu
Optimizador: rmsprop
Tasa de aprendizaje: 0.0021506750461228745
Se hace la evaluación:
102/102 ————— 0s 311us/step - accuracy: 1.0000 - loss: 1.9928e-10

##### Resultados obtenidos tras la evaluación #####
Precisión del modelo tras evaluación: 1.0
Pérdida del modelo tras evaluación: 5.001511982349882e-10
```

Figura 12. Resultado de la ejecución del código para el *dataset* Mushroom

4.2 Resultados sobre los *datasets* de prueba

4.2.1 Ajuste de los hiperparámetros de la librería de *autotune*

La librería que se ha diseñado en este proyecto tiene en sí misma varios hiperparámetros que son críticos para su funcionamiento, siendo los principales el número de trials (número de combinaciones de parámetros diferentes que se van a ejecutar cada vez que se intente optimizar un hiperparámetro o un conjunto de hiperparámetros) y el número de épocas de entrenamiento. Cuanto mayor sea el número de épocas, mayor certeza tendremos sobre el rendimiento del modelo que se está evaluando. Y cuanto mayor sea el número de trials, mayor será el número de combinaciones de hiperparámetros que se exploren y mejor será el rendimiento del modelo final. Por la contra, cuanto mayor sea el número de trials y de épocas, más tiempo de ejecución requerirá la librería para ejecutar todas las pruebas. Se hace por tanto necesario llegar a un compromiso entre el tiempo de ejecución, y la calidad de los resultados. Para ello se

han ejecutado una serie de pruebas sobre los conjuntos de datos Bank Marketing, Breast Cancer Wisconsin, Car Evaluation, Census Income, Default of Credit Card Clients, Dry Bean, Heart Disease, MAGIC Gamma Telescope, Mushroom y Spambase (ver Tabla 7). De esta manera, se puede estudiar la evolución del rendimiento y de las arquitecturas de los modelos obtenidos, en función del número de *trials* y épocas, variables que están relacionadas directamente con comportamiento del algoritmo de búsqueda de los hiperparámetros.

En cuanto a las arquitecturas obtenidas, es importante empezar destacando un comportamiento: en el caso en que el *autotune* no consiga unos hiperparámetros óptimos, lo cual provoque que la pérdida no se estabilice y no se produzca el *EarlyStopping*, el entrenamiento final del modelo puede consumir un mayor número de recursos, incluso llegando a entrenar el modelo con las 1000 épocas de límite de entrenamiento. Este suceso ocurre con una mayor probabilidad cuánto menor número de *trials* o menor número de épocas de entrenamiento por *trial* se utilizan, como se puede observar claramente en la Tabla 14, la Tabla 20 o la Tabla 22. Es decir, en estos casos, el modelo no tiene una buena arquitectura al terminar la búsqueda, por lo que su única opción para obtener unos buenos resultados finales es entrenar durante un gran tiempo. Esta no es la situación óptima que se desea conseguir como científico de datos con un modelo, ya que, en el caso de tener un *dataset* muy grande, puede que no se disponga de tal tiempo de entrenamiento o que el modelo no acabe dando el resultado esperado.

Otro aspecto importante sobre las arquitecturas obtenidas es el hecho de que, dado que la mayoría de los *datasets* utilizados no son excesivamente grandes ni complejos, existen varias alternativas de arquitecturas que funcionan correctamente para cada uno, por lo que, en muchos casos no se observa que, al aumentar el número de *trials* por etapa de búsqueda o el número de épocas de entrenamiento por *trial*, el modelo converja claramente hacia una arquitectura determinada. Además, a este aspecto también le afecta el hecho de hacer una búsqueda secuencial por etapas para encontrar los hiperparámetros, ya que, como se necesita fijar algunos valores de la arquitectura del modelo en cada etapa de búsqueda, esto provoca una menor precisión a la hora de decidir qué valores de hiperparámetros escoger. Las diferentes arquitecturas seleccionadas, aunque no converjan en una única, funcionan correctamente dando resultados aceptables

en función de los recursos utilizados, como se puede consultar en las gráficas de precisión y pérdidas comentadas de las Figuras 13-16.

La mejor arquitectura encontrada para cada modelo se establece en función del número de épocas de entrenamiento por cada *trial* y el número de *trials* por cada etapa de búsqueda (es decir, las dos variables del *tuner* con las que se estudiará la evolución de los modelos). Se puede ver que los optimizadores que suelen funcionar mejor son *rmsprop* y *adam*, con una tendencia a escoger *adam* según las variables del *tuner* aumentan en valor (ver Tabla 9-28 en el Anexo). El optimizador *sgd*, que es el último optimizador por defecto, es muy poco utilizado, por lo que para estos conjuntos parece no tener buenos resultados al entrenarse los modelos con él.

El número de capas ocultas varía mucho entre ejecuciones del mismo *dataset* con distintos valores de las variables del *tuner*, debido a que en la etapa de búsqueda del número de capas ocultas del algoritmo también se busca una primera aproximación de la tasa de aprendizaje, por lo que esta modifica en gran medida el comportamiento de la red y provoca que, por ejemplo, con 8 épocas de entrenamiento por cada *trial* se usen 6 capas ocultas y, con 64 épocas de entrenamiento por cada *trial* se usen 2 capas ocultas, como se observa en la Tabla 11.

El número de neuronas por cada capa oculta depende de la estructura del *dataset*, ya que la estructura define el valor máximo del rango de búsqueda del hiperparámetro, mediante la fórmula explicada en la sección 3.3, por lo que varía bastante entre *datasets* diferentes. Ahora bien, entre ejecuciones de un mismo *dataset* durante la evolución de las variables del *tuner*, el número de neuronas por capa oculta se mantiene relativamente estable, no desviándose normalmente respecto al valor que utiliza el primer modelo (el que menos recursos del *tuner* utiliza), como se observa, por ejemplo, en la Tabla 12 y en la Tabla 14. Todas las funciones de activación empleadas (*silu*, *elu* y *relu*) parecen tener resultados buenos, debido a que realmente *silu* y *elu* son variaciones de *relu*, por lo que la red no experimenta un gran cambio entre ellas. También debe tenerse en cuenta que este hiperparámetro suele proporcionar cambios bastante incrementales en el rendimiento de una arquitectura. Por último, se observa que la tasa de aprendizaje normalmente no se queda excesivamente cerca del límite superior: $1e^{-2}$, estableciendo valores entre algunos

órdenes de magnitud más pequeños, incluso en algunos casos llegando a valores muy pequeños del orden de $1e^{-6}$ o inferiores.

Con referencia al rendimiento de los modelos resultantes tras el *autotune*, se estudiará la evolución de la pérdida y precisión de estos, en función de las dos variables comentadas anteriormente: el número de épocas de entrenamiento por cada *trial* y el número de *trials* por cada etapa de búsqueda. Cabe resaltar que, para mayor fiabilidad de los resultados, se han corrido las pruebas tres veces con cada *dataset* haciendo una ejecución idéntica para así obtener la media y la desviación típica de los resultados del rendimiento de las tres ejecuciones.

Primero, se estudiará la evolución de los resultados si solamente se modifica el valor del número de épocas de entrenamiento por cada *trial*. Para ello, se fija el número de *trials* que se ejecutan en cada etapa de búsqueda a 5. Es decir, dado que se cuenta con 6 etapas de búsqueda en el algoritmo, en total se procesan 30 *trials*. En la gráfica de pérdida mostrada en la Figura 13, se muestra que, para un valor de 5 épocas de entrenamiento por cada *trial*, los resultados para estos *datasets* son buenos. Por tanto, para estos *datasets*, no tiene sentido aumentar el número de épocas de entrenamiento por cada *trial* por encima de 5. Este comportamiento es dependiente de los conjuntos de datos utilizados, por lo que, si se usan otros más complejos, este valor de 5 épocas podría aumentar, ya que al algoritmo le podría costar más trabajo encontrar una arquitectura compatible con el conjunto de datos. En referencia a la gráfica de precisión mostrada en la Figura 14, se observa el mismo comportamiento que para la pérdida, dando lugar a unas conclusiones muy parecidas, pero con la distinción de que existe una variación máxima sufrida por el *dataset* representado en color naranja (Breast Cancer Wisconsin Diagnostic) que es bastante significativa, ya que pasa de una precisión de 1 a una precisión de 0,6, por lo que, para ese conjunto de datos, sí que merecería la pena aumentar el número de épocas de entrenamiento por *trial*, con el objetivo de ver cómo se estabiliza el modelo, en el caso de necesitar una precisión más cercana a 1.

Para estudiar la evolución de los modelos en función del número de *trials* utilizados en cada etapa de búsqueda se fijan el número de épocas de entrenamiento por cada *trial* a 5. El comportamiento es bastante parecido a la evolución en función de la otra variable del tuner, pero con unos buenos resultados alrededor de los 3 *trials* por etapa de

búsqueda (ver Figura 15 y Figura 16), es decir, lo que equivaldría a una ejecución completa de 18 *trials* (3 *trials* por etapa x 6 etapas de búsqueda), siendo innecesario continuar aumentando el número de *trials*. Cabe destacar, que, al igual que ocurría anteriormente, si los *datasets* se cambian, el comportamiento de la precisión y la pérdida cambiará por completo, ya que este comportamiento depende de la complejidad del conjunto de datos para la búsqueda de hiperparámetros. En definitiva, en la librería se han establecido a 5, como valor por defecto, el número de *trials* y el número de épocas de entrenamiento por cada *trial*, ya que se observan buenos resultados para los conjuntos de prueba.

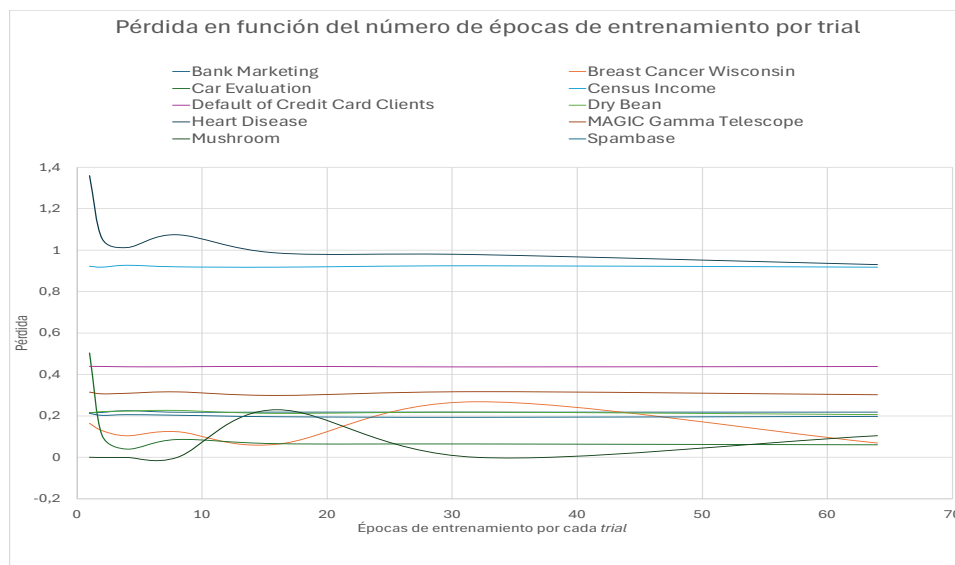


Figura 13. Gráfica de pérdida en función del número de épocas de búsqueda.

Haciendo énfasis en el *dataset* con mayor pérdida (Heart Disease), este no siempre mejora según se incrementa el número de *trials* usado. Imaginemos que se incrementa el número de *trials*, es decir, de pruebas de valores que se realizan en cada búsqueda, de manera que se podría pensar que el modelo resultante debería ser mejor en todos los puntos de las gráficas, pero esto no sucede siempre en la práctica. El método de búsqueda bayesiana, al igual que el *HyperBand*, no asegura que el mejor resultado de una iteración de $k + 1$ *trials* sea mejor que la de k *trials*, ya que solo aspira a conseguir mejorar la media de su distribución a largo plazo. Por tanto, puede ocurrir que, dado que se exploran un número de *trials* muy pequeño en comparación con el número de combinaciones posibles de hiperparámetros, si el *tuner* explora una zona de valores que

no son prometedores, no se conseguirá un buen valor para el hiperparámetro. En cambio, si la zona de exploración de valores es prometedora, con unos pocos *trials* se pueden conseguir resultados que den lugar a un buen rendimiento del modelo.

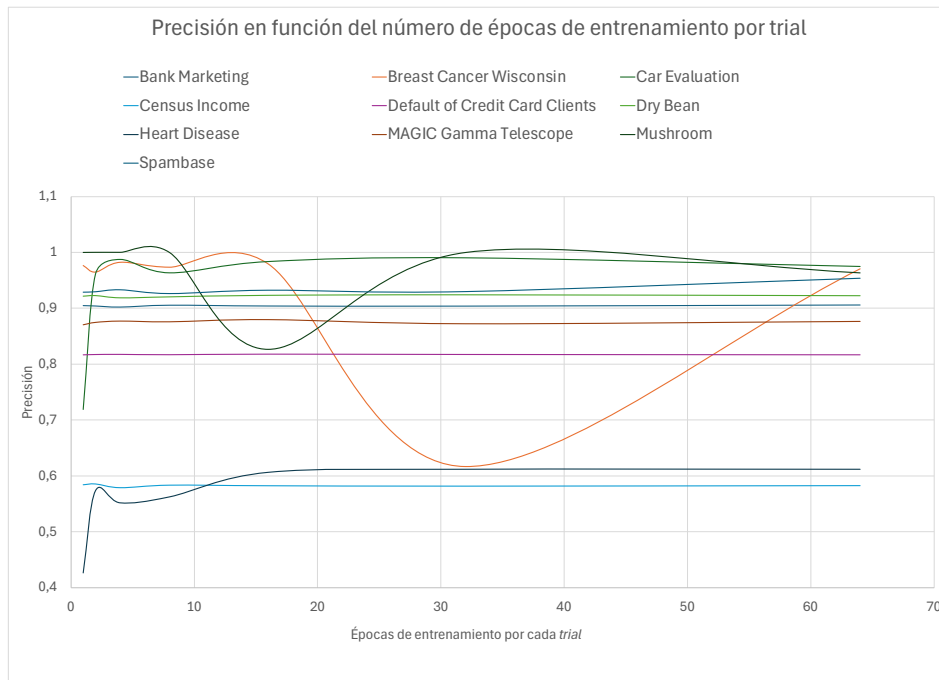


Figura 14. Gráfica de precisión en función del número de épocas de búsqueda.

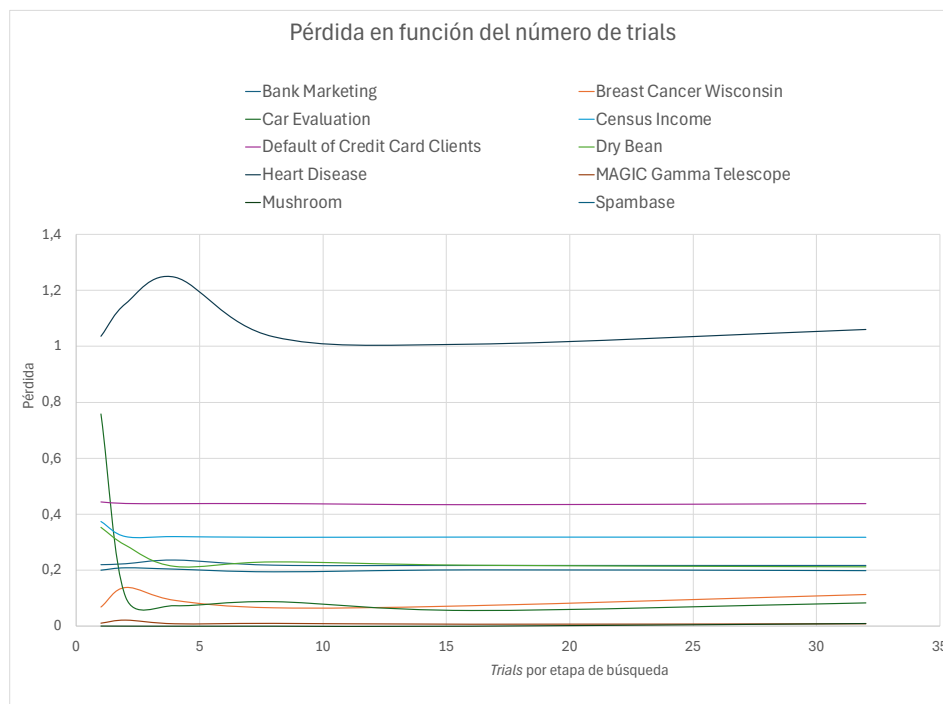


Figura 15. Gráfica de pérdida en función del número de trials.

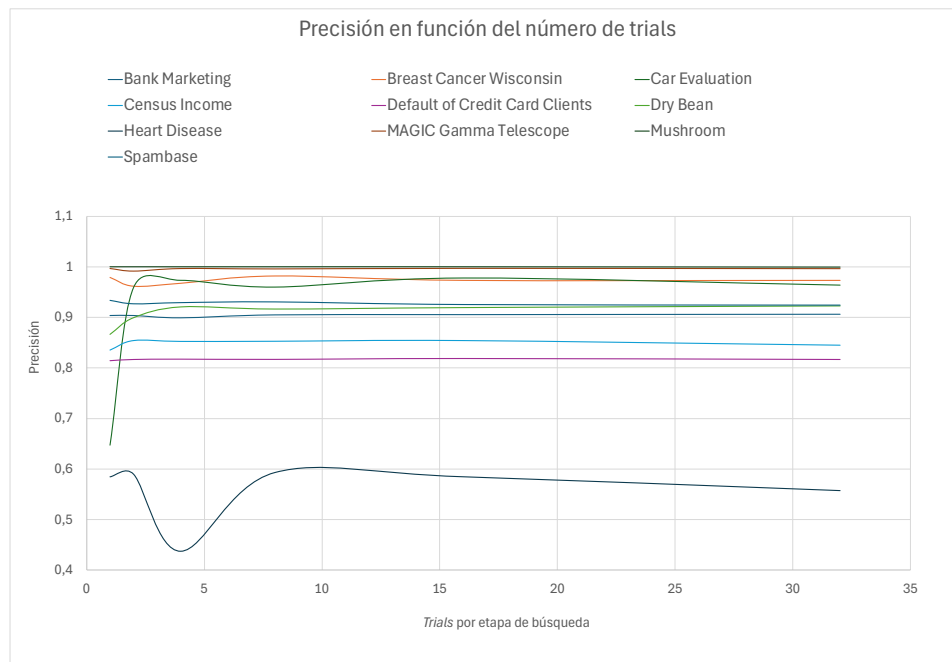


Figura 16. Gráfica de precisión en función del número de trials.

4.2.2 Evaluación del rendimiento de la librería

A continuación, se estudiará más en detalle el caso concreto de algunos *datasets* de prueba para evaluar el rendimiento de la librería. En la Tabla 8 se observan las arquitecturas de los mejores modelos obtenidos para cada *dataset* de prueba, así como los rendimientos asociados a ellos. En cuanto al *dataset* Mushroom, este parte de unas buenas características para ser entrenado, como una distribución casi por igual de las clases (51,8% y 48,2%). Según ucimlrepo (ver Tabla 7), un modelo de red neuronal de clasificación sobre este *dataset* puede llegar a 100% de precisión, lo cual se consigue utilizando una arquitectura de 5 capas ocultas con 79 neuronas por capa. Por lo tanto, la librería ha conseguido llegar al rendimiento óptimo que se ilustra en ucimlrepo utilizando solamente 5 *trials* por etapa de búsqueda y 2 épocas de entrenamiento por *trial*.

En cuanto al *dataset* Spambase, este tiene una máxima precisión alcanzable en ucimlrepo en torno al 92%, valor al que se aproxima el modelo conseguido por la librería, que es de un 95% en media. Es posible que el *dataset* haya sido procesado de distinta manera a como lo hace el modelo que obtuvo la métrica de 92,3% en ucimlrepo, de

manera que el modelo obtenido puede tener ciertas variaciones de precisión respecto a la máxima precisión que ucimlrepo expone. También es posible que exista sobreajuste en el modelo generado, provocando que el modelo aprenda de memoria los resultados, lo cual genera una mayor precisión, pero que también generalice peor ante datos que no haya estudiado anteriormente. La arquitectura del modelo para Spambase tiene 5 capas ocultas con 31 neuronas en cada una, además de utilizar el optimizador *sgd*, que, a diferencia de para el resto de *datasets*, ha dado buenos resultados. La librería ha necesitado 64 épocas de entrenamiento por *trial* con 5 *trials* por etapa de búsqueda para llegar a obtener esta arquitectura.

El *dataset* Car Evaluation también ha generado un modelo con un rendimiento muy alto de un 99% de precisión en media, superando al rendimiento aproximado de ucimlrepo de un 98,3%, usando 5 *trials* y 32 épocas de entrenamiento por *trial*. La arquitectura que genera este rendimiento está compuesta por 5 capas ocultas con 18 neuronas cada una, utilizando *rmsprop* como optimizador.

En cuanto al *dataset* Breast Cancer Wisconsin, con una máxima precisión alcanzable según ucimlrepo de 92,3%, ha obtenido un rendimiento del 98,2% en media, con una arquitectura de 5 capas ocultas y 18 neuronas por capa y *adam* como optimizador, la cual es bastante parecida a la arquitectura generada para el *dataset* Car Evaluation. Este resultado ha sido obtenido con solamente 4 épocas de entrenamiento por *trial* y 5 *trials* por etapa.

Por último, el *dataset* Heart Disease ha generado un modelo con un rendimiento de 61,2% de precisión, en comparación al valor de máxima precisión en ucimlrepo de 78,9%. Ha utilizado un modelo con 3 capas ocultas con 5 neuronas por capa, es decir, una red neuronal bastante sencilla comparado con las anteriores, necesitando 32 épocas de entrenamiento por *trial* y 5 *trials* por etapa de búsqueda. Este *dataset* parte de varios problemas, que provocan que los modelos entrenados no consigan buenos resultados, independientemente de cómo se entrenen, como una mala distribución de clases en el *dataset* (con la clase mayoritaria estando presente en el 55% del conjunto y la clase con menos representación con un 4%) y un número de instancias muy pequeño (242 instancias con el *dataset* ya procesado).

En líneas generales estos resultados muestran que el rendimiento de la librería es correcto, devolviendo un resultado aproximado al óptimo en función de la cantidad de recursos que se dediquen a la búsqueda de hiperparámetros. Estos resultados han sido obtenidos utilizando una heurística que permite reducir en gran medida el número de combinaciones de hiperparámetros a probar, ya que los resultados comentados no utilizan un gran número de *trials* ni épocas de entrenamiento. Esto es esencial, ya que permite reducir los recursos que la librería necesita para conseguir los modelos y permite que la librería sea utilizada por usuarios que no dispongan de una gran capacidad computacional para ejecutar el código. Cabe destacar que, a mayor complejidad del *dataset*, es decir, cuánto más complejos sean los patrones que la red neuronal debe aprender para predecir nuevos valores, más necesidad de recursos necesitará la librería para aproximarse al mejor resultado que satisface la necesidad del problema en cuestión.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida Media	Desviación típica de la pérdida	Precisión Media	Desviación típica de la precisión
Bank Marketing	5	32	5	32	silu	adam	0,00301	18	0,218	0,0035	0,906	0,001
Breast Cancer Wisconsin	4	5	5	18	elu	adam	0,00346	31	0,104	0,0262	0,982	0
Car Evaluation	32	5	5	18	elu	rmsprop	0,01027	50	0,064	0,0721	0,99	0,006
Census Income	5	2	5	39	elu	adam	6,60E-04	22	0,32	0,0034	0,855	0,0003
Default of Credit Card Clients	5	16	5	13	elu	adam	0,00315	16	0,434	0,0007	0,819	0,0031
Dry Bean	32	5	5	16	relu	adam	0,0003	39	0,218	0,013	0,924	0,0039
Heart Disease	32	5	3	5	elu	rmsprop	0,00483	17	0,979	0,0347	0,612	0,0341
MAGIC Gamma Telescope	5	16	2	5	elu	adam	0,00381	23	0,007	0,0035	0,997	0,002
Mushroom	2	5	5	79	silu	rmsprop	1,09E-03	13	0	0	1	0
Spambase	64	5	5	31	elu	sgd	2,57E-03	21	0,198	0,0034	0,954	0,0343

Tabla 8. Mejores resultados de la librería para cada dataset de prueba.

5 CONCLUSIONES Y TRABAJO FUTURO

En este proyecto se ha desarrollado una librería de AutoML en Keras basada en una heurística de búsquedas secuenciales, que pretende simplificar la construcción de modelos de clasificación basados en redes neuronales por parte de usuarios con poca experiencia en el ajuste de hiperparámetros. La librería sólo requiere como entrada el conjunto de datos proporcionado por el usuario. A la vez, tiene flexibilidad para que el usuario establezca rangos o valores fijos para la búsqueda de hiperparámetros en función de su conocimiento sobre la materia. El algoritmo de búsqueda se divide en varias fases: selección del número de capas, número de neuronas por cada capa, función de activación, optimizador, tasa de aprendizaje y ajuste de los hiperparámetros internos del optimizador. Además, se han incorporado técnicas de regularización para evitar el sobreajuste del modelo, como el *EarlyStopping* y la utilización de un conjunto de validación.

Con respecto a los resultados obtenidos sobre los conjuntos de datos de prueba, estos sugieren que un número de *trials* por etapa de búsqueda o épocas de entrenamiento por *trial* bajo en torno a 3 o 5 es suficiente para obtener unos resultados aceptables con este tipo de datasets, llegando a obtener resultados comparables con los mejores resultados reportados en la literatura para ese conjunto de datos según ucimlrepo.

Si los rangos de búsqueda de los hiperparámetros especificados por el usuario fuesen muy amplios, el *tuner* tendría más posibilidades de encontrar valores extremos dentro del rango de búsqueda. En ese caso, la solución para probar con un rango de valores más amplio y encontrar mejores valores es aumentar el número de *trials* de cada etapa de búsqueda o el número de épocas de entrenamiento por cada *trial*, o ambas, teniendo como objetivo. De este modo habrá más oportunidades de que el algoritmo del *tuner* pueda encontrar buenos valores y profundizar en ellos. No obstante, esto conlleva que la librería emplee una mayor cantidad de recursos.

En relación con el algoritmo interno de búsqueda de la librería, es importante destacar que existe un compromiso entre el tipo de búsqueda que se realiza y los recursos utilizados, entendiendo tipo de búsqueda como la diferencia entre hacer una única búsqueda que abarque todo el espacio de valores de hiperparámetros y hacer varias búsquedas secuenciales que disminuyan en gran cantidad el número de combinaciones

posibles a probar. Si se requiere un tiempo de computación o procesamiento relativamente pequeño, es necesario optar por la segunda alternativa, las búsquedas secuenciales, tal como se hace en esta librería, ya que, si no, el número de combinaciones a probar se vuelve exponencial, lo cual es inabarcable si se dispone de unos recursos limitados. Dicho esto, debe tenerse en cuenta que los distintos hiperparámetros de los modelos interaccionan entre ellos. Por ello, la aproximación de pruebas secuencial es subóptima y corre un riesgo mayor de caer en mínimos locales del espacio de hiperparámetros.

Como trabajo futuro, se plantea extender el estudio del algoritmo de *autotune* de esta librería para la utilización de recursos inferiores a la época de búsqueda, permitiendo analizar la evolución y comportamiento de los modelos para unos pocos *batches* de búsqueda en cada *trial*, con el objetivo de encontrar aproximaciones de los resultados con muy pocos recursos. Asimismo, dado que los conjuntos de prueba utilizados han demostrado ser bastante sencillos, ya que se pueden modelar correctamente con distintas arquitecturas, se propone evaluar la herramienta para conjuntos de datos más complejos, incluso con varios millones de registros de entrenamiento, para entender qué estrategias se pueden llevar a cabo para reducir los recursos y mejorar el algoritmo de *autotune* empleado, permitiendo generalizar las conclusiones que se pueden extraer de la librería. También se podría introducir la posibilidad de realizar una validación basada en pliegues que permita tener una mejor aproximación acerca de los resultados sobre los modelos finales generados. Esto requeriría una mayor cantidad de recursos computacionales, pero incrementaría la fiabilidad de las métricas de rendimiento de los modelos. Por último, sería interesante estudiar la viabilidad de extender la librería para incorporar soluciones a problemas de regresión, dado que actualmente solo da soporte para problemas de clasificación genéricos.

6 BIBLIOGRAFÍA

- [1] Norvig, P. R. (2021). *ARTIFICIAL INTELLIGENCE: A Modern Approach, Global Edition*.
- [2] Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill Science/Engineering/Math.
- [3] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and Tensorflow: Concepts, Tools, and Techniques to Build Intelligent Systems*.
- [4] Holdsworth, J., & Scapicchio, M. (2025, 21 febrero). Deep learning. *IBM Think*. Recuperado 10 de junio de 2025, de <https://www.ibm.com/es-es/think/topics/deep-learning>
- [5] LeCun, Y., Bengio, Y., & Hinton, G. (2015). *Deep Learning*. *Nature*, 521(7553), 436-444.
- [6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- [7] Universidad de Sevilla (s.f.). *Redes neuronales multicapa* [Diapositivas]. <https://biblus.us.es/bibing/proyectos/abreproy/70184/fichero/Capitulo5.pdf>
- [8] Moez, A. (2024, 15 abril). Introducción a las funciones de activación en las redes neuronales. *Datacamp*. <https://www.datacamp.com/es/tutorial/introduction-to-activation-functions-in-neural-networks>
- [9] EITCA. (2023, 13 agosto). ¿Cuál es el papel del optimizador en el entrenamiento de un modelo de red neuronal? *EITCA*. <https://es.eitca.org/artificial-intelligence/eitc-ai-dlpp-deep-learning-with-python-and-pytorch/neural-network/training-model-neural-network/examination-review-training-model-neural-network/what-is-the-role-of-the-optimizer-in-training-a-neural-network-model/>
- [10] Jeeva, C. (2023, 9 mayo). Momentum-Based gradient descent. *Scaler*. <https://www.scaler.com/topics/momentum-based-gradient-descent/>
- [11] Kashyap, P. (2024, 2 noviembre). Understanding Nesterov Accelerated Gradient (NAG). *Medium*. <https://medium.com/%40piyushkashyap045/understanding-nesterov-accelerated-gradient-nag-340c53d64597>
- [12] Team, K. (s. f.). *Keras Documentation: Adam*. Recuperado 14 de junio de 2025, de <https://keras.io/api/optimizers/adam/>
- [13] Team, K. (s. f.-b). *Keras documentation: KerasTuner*. https://keras.io/keras_tuner/
- [14] Bhumildobariya. (2023, 29 octubre). AutoKeras: Revolutionizing Machine Learning with Python. *Medium*. <https://medium.com/%40bhumildobariya88/autokeras-revolutionizing-machine-learning-with-python-dd289c0d22ff>
- [15] *Built-in developer tools - Features | PyCharm*. (s. f.). JetBrains. Recuperado 22 de mayo de 2025, de <https://www.jetbrains.com/pycharm/features/tools.html>
- [16] *About Git - GitHub Docs*. (s. f.). GitHub Docs. Recuperado 22 de mayo de 2025, de <https://docs.github.com/en/get-started/using-git/about-git/>
- [17] *venv — Creation of virtual environments*. (s. f.). Python Documentation. Recuperado 23 de mayo de 2025, de <https://docs.python.org/3/library/venv.html>
- [18] *User guide — conda 25.5.2.dev31 documentation*. (s. f.). Recuperado 23 de mayo de 2025, de <https://docs.conda.io/projects/conda/en/latest/user-guide/>
- [19] *conda-forge | community-driven packaging for conda*. (s. f.). Recuperado 23 de mayo de 2025, de <https://conda-forge.org/>
- [20] Team, K. (s. f.-c). *Keras documentation: KerasTuner HyperModels*. Recuperado 17 de junio de 2025, de https://keras.io/keras_tuner/api/hypermodels/
- [21] Team, K. (s. f.-d). *Keras documentation: KerasTuner Oracles*. Recuperado 17 de junio de 2025, de https://keras.io/keras_tuner/api/oracles/
- [22] Team, K. (s. f.-e). *Keras documentation: The Tuner classes in KerasTuner*. Recuperado 17 de junio de 2025, de https://keras.io/keras_tuner/api/tuners/
- [23] *UCI Machine Learning Repository*. (s. f.). Recuperado 20 de junio de 2025, de <https://archive.ics.uci.edu/dataset/45/heart+disease>
- [24] *UCI Machine Learning Repository*. (s. f.-b). Recuperado 20 de junio de 2025, de <https://archive.ics.uci.edu/dataset/73/mushroom>

- [25] *UCI Machine Learning Repository.* (s. f.-c). Recuperado 20 de junio de 2025, de <https://archive.ics.uci.edu/dataset/94/spambase>
- [26] *UCI Machine Learning Repository.* (s. f.-d). Recuperado 20 de junio de 2025, de <https://archive.ics.uci.edu/dataset/19/car+evaluation>
- [27] *UCI Machine Learning Repository.* (s. f.-e). Recuperado 20 de junio de 2025, de <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

Anexo

A continuación, se muestran los resultados de las ejecuciones de las pruebas de validación de la librería con los datasets de pruebas en formato de tabla. Para cada uno de los datasets (ver Tabla 7) se reportarán los resultados de haber fijado el número máximo de trials por etapa de búsqueda a 5 y permitiendo que varíe el valor del número de épocas de entrenamiento por *trial* de 1 a 64, así como los resultados de haber fijado el número de épocas de entrenamiento por *trial* a 5 y permitiendo que se varíe el número de *trials* por etapa de búsqueda de 1 a 32.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Bank Marketing	1	5	5	34	silu	adam	0,00502	21	0,2159	0,9048
Bank Marketing	2	5	2	25	silu	adam	0,00251	19	0,2169	0,9041
Bank Marketing	4	5	5	32	relu	adam	0,00194	20	0,2258	0,9022
Bank Marketing	8	5	3	31	silu	adam	0,00529	16	0,218	0,9055
Bank Marketing	16	5	2	23	relu	adam	0,00086	25	0,2186	0,9041
Bank Marketing	32	5	2	32	relu	adam	0,00015	64	0,2185	0,9039
Bank Marketing	64	5	6	20	silu	rmsprop	0,00052	18	0,2185	0,9059

Tabla 9. Resultados de la librería sobre el dataset Bank Marketing fijando el número máximo de trials por etapa de búsqueda.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Bank Marketing	5	1	4	12	silu	adam	0,00029	55	0,2202	0,9037
Bank Marketing	5	2	6	24	silu	adam	0,00014	46	0,2234	0,9037
Bank Marketing	5	4	3	32	elu	rmsprop	1,48E-08	1000	0,2359	0,8993
Bank Marketing	5	8	2	21	silu	adam	0,00286	22	0,2186	0,9049
Bank Marketing	5	16	2	26	relu	adam	0,00373	17	0,2176	0,9054
Bank Marketing	5	32	5	32	silu	adam	0,00301	18	0,2176	0,9061

Tabla 10. Resultados de la librería sobre el dataset Bank Marketing fijando el número de épocas de entrenamiento por trial.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Breast Cancer Wisconsin	1	5	6	21	elu	rmsprop	0,01338	19	0,1641	0,9766
Breast Cancer Wisconsin	2	5	6	15	elu	adam	2,06E-06	1000	0,1292	0,9649
Breast Cancer Wisconsin	4	5	5	18	elu	adam	0,00346	31	0,1042	0,9824
Breast Cancer Wisconsin	8	5	6	16	elu	rmsprop	0,00174	11	0,1239	0,9736
Breast Cancer Wisconsin	16	5	5	11	relu	adam	0,00155	22	0,0637	0,9795
Breast Cancer Wisconsin	32	5	4	18	elu	adam	0,00031	49	0,268	0,6169
Breast Cancer Wisconsin	64	5	2	5	relu	rmsprop	0,00072	24	0,0694	0,9707

Tabla 11. Resultados de la librería sobre el dataset Breast Cancer Wisconsin fijando el número máximo de trials por etapa de búsqueda.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Breast Cancer Wisconsin	5	1	2	21	elu	rmsprop	0,00019	49	0,069	0,9795
Breast Cancer Wisconsin	5	2	2	14	relu	rmsprop	4,22E-07	1000	0,1378	0,9619
Breast Cancer Wisconsin	5	4	5	18	relu	adam	1,58E-03	23	0,0924	0,9678
Breast Cancer Wisconsin	5	8	3	21	elu	adam	0,00233	14	0,0663	0,9824
Breast Cancer Wisconsin	5	16	6	20	elu	rmsprop	0,0044	20	0,0736	0,9736
Breast Cancer Wisconsin	5	32	2	22	silu	adam	0,00775	18	0,1131	0,9736

Tabla 12. Resultados de la librería sobre el dataset Breast Cancer Wisconsin fijando el número de épocas de entrenamiento por trial.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Car Evaluation	1	5	3	17	silu	sgd	2,70E-08	11	0,504	0,7188
Car Evaluation	2	5	5	14	elu	adam	4,61E-02	21	0,1062	0,9614
Car Evaluation	4	5	3	18	silu	adam	0,00822	29	0,0395	0,9874
Car Evaluation	8	5	5	15	elu	adam	0,00067	76	0,0859	0,9634
Car Evaluation	16	5	3	7	silu	rmsprop	0,01568	19	0,0657	0,9836
Car Evaluation	32	5	5	18	elu	rmsprop	0,01027	50	0,0644	0,9903
Car Evaluation	64	5	4	15	elu	adam	0,001	85	0,0603	0,9748

Tabla 13. Resultados de la librería sobre el dataset Car Evaluation fijando el número máximo de trials por etapa de búsqueda.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Car Evaluation	5	1	2	13	silu	sgd	5,16E-06	1000	0,7583	0,6472
Car Evaluation	5	2	4	16	elu	rmsprop	2,88E-05	300	0,1047	0,9595
Car Evaluation	5	4	3	18	silu	adam	3,80E-04	143	0,0737	0,9737
Car Evaluation	5	8	6	14	elu	rmsprop	0,00317	42	0,0881	0,9603
Car Evaluation	5	16	2	18	relu	adam	0,01055	36	0,0566	0,9783
Car Evaluation	5	32	2	15	silu	rmsprop	0,03681	11	0,0837	0,9642

Tabla 14. Resultados de la librería sobre el dataset Car Evaluation fijando el número de épocas de entrenamiento por trial.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Census Income	1	5	5	52	relu	adam	7,50E-04	16	0,9226	0,5841
Census Income	2	5	5	46	silu	adam	1,04E-03	19	0,9184	0,5855
Census Income	4	5	5	43	relu	rmsprop	0,00079	16	0,927	0,579
Census Income	8	5	2	48	relu	adam	0,00019	19	0,92	0,5834
Census Income	16	5	3	67	silu	adam	0,00042	18	0,9184	0,5824
Census Income	32	5	3	11	elu	rmsprop	0,00146	18	0,9247	0,5817
Census Income	64	5	2	22	silu	adam	3,74E-05	98	0,9183	0,5826

Tabla 15. Resultados de la librería sobre el dataset Census Income fijando el número máximo de trials por etapa de búsqueda.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Census Income	5	1	4	51	silu	rmsprop	4,32E-03	12	0,3743	0,8357
Census Income	5	2	5	39	elu	adam	6,60E-04	22	0,3199	0,8546
Census Income	5	4	4	65	relu	adam	6,50E-04	15	0,3199	0,8529
Census Income	5	8	6	52	relu	adam	0,00039	15	0,3173	0,8532
Census Income	5	16	2	61	silu	adam	0,0002	18	0,3181	0,8546
Census Income	5	32	6	24	relu	adam	0,00094	16	0,3174	0,8454

Tabla 16. Resultados de la librería sobre el dataset Census Income fijando el número de épocas de entrenamiento por trial.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Default of Credit Card Clients	1	5	6	13	silu	adam	1,17E-03	15	0,4389	0,8166
Default of Credit Card Clients	2	5	3	17	silu	rmsprop	2,68E-03	16	0,4385	0,8173
Default of Credit Card Clients	4	5	2	18	silu	rmsprop	0,00431	14	0,4375	0,8176
Default of Credit Card Clients	8	5	6	16	silu	adam	0,001	25	0,4374	0,8169
Default of Credit Card Clients	16	5	4	14	elu	rmsprop	0,00209	17	0,4387	0,8181
Default of Credit Card Clients	32	5	3	10	elu	adam	0,0006	36	0,4371	0,8174
Default of Credit Card Clients	64	5	5	16	elu	adam	1,30E-04	39	0,4383	0,8167

Tabla 17. Resultados de la librería sobre el dataset Default of Credit Card Clients fijando el número máximo de trials por etapa de búsqueda.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Default of Credit Card Clients	5	1	5	5	elu	rmsprop	1,02E-05	96	0,4446	0,8143
Default of Credit Card Clients	5	2	2	11	elu	adam	7,78E-06	800	0,4388	0,8169
Default of Credit Card Clients	5	4	4	18	elu	adam	9,77E-06	296	0,4382	0,8176
Default of Credit Card Clients	5	8	5	18	elu	adam	0,0016	20	0,4385	0,8171
Default of Credit Card Clients	5	16	5	13	elu	adam	0,00315	16	0,4343	0,819
Default of Credit Card Clients	5	32	6	18	silu	adam	0,00187	18	0,4383	0,8169

Tabla 18. Resultados de la librería sobre el dataset Default of Credit Card Clients fijando el número de épocas de entrenamiento por trial.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Dry Bean	1	5	3	17	elu	adam	2,10E-04	67	0,2144	0,9217
Dry Bean	2	5	6	12	relu	adam	4,07E-03	31	0,221	0,9228
Dry Bean	4	5	4	15	elu	rmsprop	0,00161	25	0,2232	0,9186
Dry Bean	8	5	2	17	silu	rmsprop	0,00302	11	0,2265	0,9204
Dry Bean	16	5	2	18	silu	adam	5,20E-05	164	0,212	0,9233
Dry Bean	32	5	5	16	relu	adam	0,0003	39	0,2179	0,9243
Dry Bean	64	5	2	14	elu	adam	3,50E-04	59	0,2062	0,9226

Tabla 19. Resultados de la librería sobre el dataset Dry Bean fijando el número máximo de trials por etapa de búsqueda.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Dry Bean	5	1	5	10	relu	rmsprop	2,60E-07	1000	0,3527	0,8668
Dry Bean	5	2	5	12	elu	rmsprop	8,67E-08	1000	0,2889	0,8998
Dry Bean	5	4	6	18	elu	adam	4,80E-04	29	0,2133	0,9211
Dry Bean	5	8	2	10	silu	rmsprop	0,00785	11	0,2294	0,9169
Dry Bean	5	16	3	14	silu	rmsprop	0,00378	13	0,2175	0,9198
Dry Bean	5	32	2	15	silu	adam	0,00544	24	0,2119	0,9231

Tabla 20. Resultados de la librería sobre el dataset Dry Bean fijando el número de épocas de entrenamiento por trial.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Heart Disease	1	5	2	13	silu	rmsprop	6,69E-05	111	1,3603	0,4262
Heart Disease	2	5	5	13	elu	rmsprop	5,64E-03	14	1,0566	0,5737
Heart Disease	4	5	4	12	elu	adam	0,00016	121	1,0133	0,5518
Heart Disease	8	5	6	6	silu	rmsprop	0,01596	12	1,075	0,5625
Heart Disease	16	5	2	12	elu	rmsprop	1,59E-03	13	0,987	0,6065
Heart Disease	32	5	3	5	elu	rmsprop	0,00483	17	0,9791	0,612
Heart Disease	64	5	4	8	silu	adam	6,24E-03	15	0,9309	0,612

Tabla 21. Resultados de la librería sobre el dataset Heart Disease fijando el número máximo de trials por etapa de búsqueda.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Heart Disease	5	1	5	9	silu	adam	1,31E-05	1000	1,0359	0,5846
Heart Disease	5	2	6	14	elu	adam	2,22E-06	1000	1,1527	0,5898
Heart Disease	5	4	3	7	silu	rmsprop	6,19E-03	13	1,2473	0,4371
Heart Disease	5	8	6	13	silu	adam	0,04494	15	1,034	0,5931
Heart Disease	5	16	6	13	silu	adam	0,01326	14	1,0075	0,5847
Heart Disease	5	32	2	14	relu	adam	0,00988	15	1,0602	0,5573

Tabla 22. Resultados de la librería sobre el dataset Heart Disease fijando el número de épocas de entrenamiento por trial.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
MAGIC Gamma Telescope	1	5	2	9	silu	adam	5,50E-03	60	0,3149	0,8705
MAGIC Gamma Telescope	2	5	4	8	silu	adam	9,80E-04	59	0,3068	0,8749
MAGIC Gamma Telescope	4	5	4	9	silu	adam	0,01505	28	0,3094	0,8773
MAGIC Gamma Telescope	8	5	6	9	silu	adam	0,00212	46	0,3159	0,876
MAGIC Gamma Telescope	16	5	6	8	silu	adam	3,10E-04	167	0,2995	0,88
MAGIC Gamma Telescope	32	5	2	7	silu	rmsprop	0,01056	15	0,3168	0,8724
MAGIC Gamma Telescope	64	5	2	9	silu	adam	1,74E-03	62	0,3027	0,8767

Tabla 23. Resultados de la librería sobre el dataset MAGIC Gamma Telescope fijando el número máximo de trials por etapa de búsqueda.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
MAGIC Gamma Telescope	5	1	6	7	silu	rmsprop	1,40E-04	26	0,0108	0,9967
MAGIC Gamma Telescope	5	2	4	6	silu	rmsprop	4,44E-07	1000	0,0213	0,9916
MAGIC Gamma Telescope	5	4	2	8	elu	rmsprop	2,76E-03	15	0,0087	0,9966
MAGIC Gamma Telescope	5	8	6	7	silu	rmsprop	0,0039	19	0,0101	0,9959
MAGIC Gamma Telescope	5	16	2	5	elu	adam	0,00381	23	0,0072	0,997
MAGIC Gamma Telescope	5	32	2	8	elu	rmsprop	0,00289	16	0,0087	0,9964

Tabla 24. Resultados de la librería sobre el dataset MAGIC Gamma Telescope fijando el número de épocas de entrenamiento por trial.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Mushroom	1	5	6	68	elu	adam	1,45E-03	14	0,0012	0,9998
Mushroom	2	5	5	79	silu	rmsprop	1,09E-03	13	0	1
Mushroom	4	5	3	70	relu	adam	0,0049	11	0	1
Mushroom	8	5	6	79	elu	rmsprop	0,00535	12	0,0003	0,9997
Mushroom	16	5	4	58	relu	rmsprop	5,10E-04	13	0,23	0,8267
Mushroom	32	5	5	48	relu	adam	0,00086	15	0,0007	0,9997
Mushroom	64	5	3	57	elu	rmsprop	5,99E-02	24	0,105	0,9633

Tabla 25. Resultados de la librería sobre el dataset Mushroom fijando el número máximo de trials por etapa de búsqueda.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Mushroom	5	1	3	70	elu	adam	5,70E-07	909	0,0005	1
Mushroom	5	2	3	20	elu	adam	5,90E-06	275	0,0002	1
Mushroom	5	4	6	25	relu	adam	5,89E-03	13	0	1
Mushroom	5	8	4	42	relu	rmsprop	0,02044	11	0	1
Mushroom	5	16	5	64	relu	adam	0,01463	13	0	1
Mushroom	5	32	5	38	silu	rmsprop	0,00515	12	0,0093	0,9996

Tabla 26. Resultados de la librería sobre el dataset Mushroom fijando el número de épocas de entrenamiento por trial.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Spambase	1	5	3	39	elu	rmsprop	6,50E-03	11	0,2117	0,9286
Spambase	2	5	6	36	relu	rmsprop	5,90E-03	18	0,2028	0,9294
Spambase	4	5	4	37	elu	rmsprop	0,00146	14	0,2057	0,933
Spambase	8	5	6	36	elu	adam	0,0014	16	0,2032	0,9261
Spambase	16	5	2	17	relu	adam	7,22E-03	15	0,1963	0,9322
Spambase	32	5	2	40	silu	adam	0,00104	16	0,1939	0,9297
Spambase	64	5	5	31	elu	sgd	2,57E-03	21	0,1979	0,954

Tabla 27. Resultados de la librería sobre el dataset Spambase fijando el número máximo de trials por etapa de búsqueda.

Dataset	Número de épocas de entrenamiento por trial	Número máximo de trials por etapa de búsqueda	Número de capas ocultas	Número de neuronas por capa oculta	Función de activación	Optimizador	Tasa de aprendizaje	Número de épocas del entrenamiento final del modelo	Pérdida	Precisión
Spambase	5	1	2	12	silu	adam	1,74E-05	736	0,2003	0,9337
Spambase	5	2	3	30	elu	adam	3,77E-05	196	0,2085	0,9272
Spambase	5	4	6	12	elu	rmsprop	6,20E-03	19	0,2039	0,9294
Spambase	5	8	3	21	silu	rmsprop	0,00216	12	0,1946	0,9308
Spambase	5	16	2	40	relu	adam	0,00202	14	0,2013	0,9258
Spambase	5	32	3	31	elu	rmsprop	0,00218	16	0,1987	0,9247

Tabla 28. Resultados de la librería sobre el dataset Spambase fijando el número de épocas de entrenamiento por trial.